eman ta zabal zazu

Universidad del País Vasco
Euskal Herriko Unibertsitatea

INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

Bachelor Thesis

Bachelor's degree in Artificial Intelligence

Adaptation of Visual Reasoners Based on Code Generation Through Fine-Tuning and Preference Optimization

Josu Barrutia

Advisors
Oier López de Lacalle
Ander Salaberria

June 22, 2025

Acknowledgements

I would like to thank my advisors, Oier and Ander, for their support and guidance since the start of my internship at Hitz zentroa. Our weekly meetings helped shape this project and kept me motivated throughout. Thanks to you, I have come to realize that research is one of the things I enjoy the most.

I am also grateful to the IXA group because without access to its cluster this project would have been far more limited.

Abstract

Answering visual queries is a complex task that requires both visual processing and reasoning. End-to-end models, the dominant approach for this task, do not explicitly differentiate between the two, limiting interpretability and generalization. Learning modular programs presents a promising alternative, but has proven challenging due to the difficulty of learning both the programs and the modules simultaneously. We present a preference dataset with three variants and a dataset with gold-truth programs to fine-tune code generation models. Using these datasets, we adapt several open-source LLMs and compare two adaptation methodologies: Direct Preference Optimization and Supervised Fine-Tuning. Our results show that both methods significantly improve model performance, bringing their code generation performance closer to the theoretical upper bound defined by the collective knowledge of all models. Notably, our fine-tuned models, with only 7–8B parameters, achieve performance competitive with the original 175B model used in ViperGPT. This work demonstrates the effectiveness of fine-tuning smaller models to enhance visual reasoning capabilities through code generation.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Motivation

Visual reasoning, the process of understanding and drawing logical conclusions from visual information, remains a complex and challenging task for artificial intelligence systems. The dominant paradigm for tackling such tasks relies on end-to-end models, which attempt to map visual inputs directly to answers. While powerful, these models often operate as "black boxes", lacking the structured, compositional reasoning abilities demonstrated by humans. This monolithic approach limits their interpretability, hampers their generalization to novel scenarios, and can lead to memorization of dataset-specific biases rather than genuine understanding.

To address these shortcomings, recent frameworks like ViperGPT have emerged, proposing a novel approach that decomposes visual queries into executable Python code. By leveraging Large Language Models (LLMs) to generate programs that call specialized vision modules, this method introduces a more transparent, modular, and interpretable reasoning process. The system can break down a complex question into a sequence of verifiable sub-tasks, mirroring a step-by-step analytical procedure.

However, the original ViperGPT framework has a significant limitation in that it operates purely in a zero-shot or few-shot capacity. The program-generating LLM is guided solely by an initial prompt and cannot be further trained or adapted to a specific downstream visual reasoning task. This inability to fine-tune the model prevents it from learning from its mistakes or specializing its code generation capabilities for the target domain.

This work is motivated by the need to solve this limitation. ViperGPT is composed of two primary stages: code generation and code execution. While either stage can be a target for enhancement, our focus is on the former. We hypothesize that by creating high-quality, task-specific datasets, we can fine-tune the code-generating LLMs. These datasets are built thanks to the

execution of the generated code, which produces an answer that can be directly compared to the annotated gold-standard answers in the dataset. This comparison allows us to assess the quality of the generated code and use this signal to guide model adaptation. The objective of this thesis is therefore to develop preference-based and gold-standard datasets to enable the adaptation of these models, aiming to significantly improve their performance and reliability within the ViperGPT framework for visual reasoning.

1.2 Contributions

The primary goal of this thesis is to enhance the ViperGPT system by enabling the adaptation of its code generation models. To this end, the foremost contribution of this work is the development of two distinct types of synthetic datasets specifically designed to fine-tune LLMs for visual reasoning through code generation. The first is a preference dataset, presented in three variants, which contains pairs of generated code where one is demonstrably *preferred* over another based on a clear quality hierarchy; this is tailored for alignment techniques like Direct Preference Optimization (DPO). The second is a Supervised Fine-Tuning (SFT) dataset composed of *gold-truth* programs, where each program has been verified to execute successfully and produce the correct answer to its corresponding visual query.

Building upon these datasets, a further contribution is the comprehensive fine-tuning and evaluation conducted throughout this thesis. We performed an extensive set of experiments, fine-tuning a diverse range of open-source LLMs (including variants of Llama 3.1, CodeLlama, and Qwen2.5) using our newly created datasets. We applied and compared two distinct adaptation methodologies, DPO and SFT.

Finally, this work provides a performance analysis of the experimental outcomes. This analysis includes a comparative evaluation of the effectiveness of DPO versus SFT for this specific task, highlighting differences in generalization between validation and test sets. The performance of our fine-tuned models is benchmarked against their baseline counterparts and the original ViperGPT system, demonstrating significant improvements. Furthermore, the analysis investigates the concept of collective versus individual knowledge, assessing whether our fine-tuning strategies allow individual models to approach the aggregated performance upper-bound of the group.

1.3 Connection with the Sustainable Development Goals (SDGs)

This project is connected to the Sustainable Development Goals (SDGs) in a variety of ways. The most direct connection is to SDG 9: Industry, Innovation, and Infrastructure. The current trajectory of AI development often relies on building larger models that demand immense computational resources, a trend that is not sustainable in the long term. This project promotes innovation, Target 9.5, by developing novel synthetic datasets and fine-tuning methodologies to significantly improve the performance of smaller, more efficient open-source LLMs. By enabling 7-8 billion

parameter models to approach the performance of vastly larger proprietary systems like the 175 billion parameter model used in the original ViperGPT, this work contributes to the development of more sustainable and "environmentally sound technologies" as called for in Target 9.4. The strategic use of Low-Rank Adaptation (LoRA) is central to this effort, as it drastically reduces the computational and memory costs associated with training, making advanced AI more feasible and accessible.

1.4 Structure of the document

This bachelor thesis is structured as follows. Chapter 2 presents the necessary background information on the visual reasoning approach that has been followed, a description of the models, an overview of the adaptation methods and an explanation of the multimodal dataset used. Chapter 3 explains the different output classifications and how the preference datasets were created based on them, it also covers the creation of the SFT dataset. Chapter 4 describes the experimental setup, including the models used, the training process, the hyperparameter search and the final results obtained. Chapter 5 offers a detailed qualitative analysis of these results, evaluating the effectiveness of the different fine-tuning approaches, estimating the semantic and inference errors, and comparing the outcomes against the baseline models. Finally, Chapter 6 concludes the thesis by summarizing the key findings and suggesting potential avenues for future work.

CHAPTER 2

State of the art

2.1 Visual Reasoning

Visual reasoning is the process of understanding, interpreting, and drawing conclusions from visual information. For an artificial intelligence system, this involves being able to process and reason over a visual query.

The dominant approach for addressing visual reasoning tasks relies on end-to-end models [2, 3, 4]. Some of these models are usually built upon large pretrained language models, which are based on text and treat inputs and outputs as sequences of discrete tokens. However, tokens are discrete, and visual information comes from the real world, which, as perceived by humans, is continuous.

In addition, human intelligence demonstrates the ability to compositionally combine individual reasoning steps to understand the visual world. In contrast, end-to-end models typically lack this kind of structured compositional reasoning and operate in a less interpretable way. Ideally, a visual query should be addressed by processing the visual input in a step-by-step manner, leveraging intermediate reasoning steps and tools tailored to different subtasks. It has been shown that this step by step process can be encouraged through a technique known as Chain-of-Thought prompting [5]. This method involves modifying the prompt to explicitly instruct the model to detail its reasoning.

To address these limitations, ViperGPT [6] proposes a novel framework that integrates code generation models with vision models. This allows the system to decompose a visual query into subroutines that can be composed and executed sequentially, providing a more interpretable and flexible reasoning process.

Given a visual input x and a textual query q about its contents, a program $z = \pi(q)$ is first

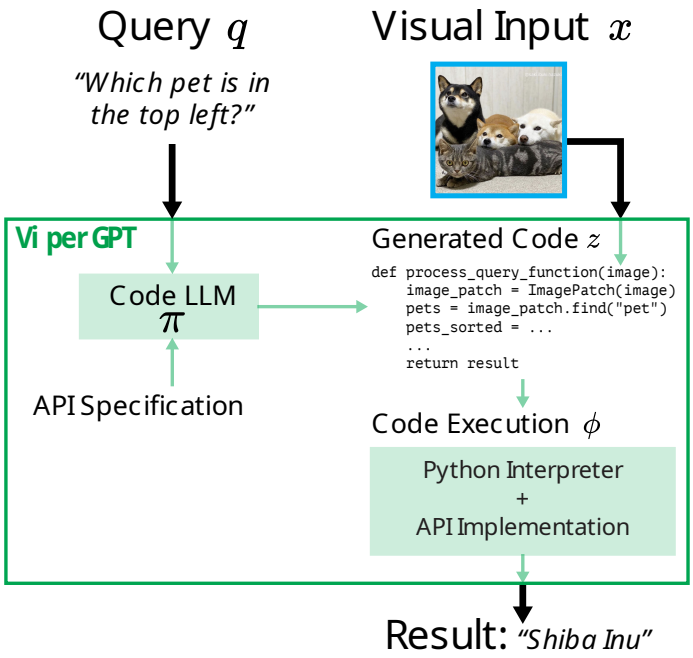

Figure 2.1: Example of the ViperGPT framework. The pipeline is divided into two different stages: a program generation stage and an execution stage. Image obtained from [6].

synthesized using a program generator π_θ, which is a neural network parameterized by θ. The execution engine ϕ then applies the program z to the input x, producing a result $r = \phi(x, z)$. This execution pipeline is illustrated in Figure 2.1.

With this approach, the main challenge arises from the fact that the outputs of $\pi_\theta(q)$ are sampled and subsequently passed as inputs to $\phi(x, z)$. Therefore, the composition $r = \phi(x, \pi_\theta(q))$ is not differentiable with respect to θ. This means gradients cannot be propagated through the entire pipeline, and standard backpropagation techniques cannot be directly applied to train π_θ. Moreover, ground-truth programs for supervision are typically unavailable and cannot be automatically collected.

To overcome this limitation, ViperGPT employs a LLM, specifically Codex [7] (code-davinci-002), as its program generator. By using Codex to generate code directly in Python, the system can benefit from training at scale from the internet, where Python code is abundant. To adapt this model for the task at hand, the model is prompted with an API that defines available vision modules, followed by a few-shot sequence and the user's query. This prompt alone enables the LLM to induce valid programs z from a given query q. The prompt used in this thesis is provided in Appendix B.

However, this approach still lacks the ability to further adapt the program generator to the specific downstream task, which remains a challenge in this line of research.

Vision Module Invocation

During program execution, the visual reasoning capabilities of the system rely on calling predefined Python functions that act as interfaces to VLMs. Each function encapsulates a specific subtask, such as object grounding, image-text matching, or visual classification. These functions are referred to as visual modules and are exposed to the LLM via the initial API definition in the prompt.

Below is a summary of the main functions used to call visual modules in the ViperGPT execution engine:

(a) `simple_query(image, query)`: Handles basic questions that require a direct answer from an image, such as "What animal is this?" Internally, this function uses the BLIP-2 model.

(b) `best_text_match(phrases, image)`: Given a list of phrases and one image, it returns the phrase that best describes the image. It uses the X-VLM model.

(c) `find(image, noun_phrase)`: Returns a list of bounding boxes (or patches) in the image corresponding to a given noun phrase. This module uses the GLIP model for object grounding.

(d) `exists(image, noun_phrase)`: Returns a boolean indicating whether the noun phrase is present in the image. It also relies on GLIP.

These functions represent the modular interface between the code generation model and pretrained vision-language models. Each function corresponds to a distinct visual processing capability and is invoked as part of the generated Python program. A full list of available modules and their usage patterns is included in Appendix B.

2.2 Models

In this section, we review the models relevant to our work. We focus on the Large Language Models (LLMs) used as program generators and the Vision-Language Models invoked by the execution engine to process visual inputs.

2.2.1 Large Language Models

LLMs are designed to understand and generate human language. They take as input a tokenized sequence of text (a *prompt*) and autoregressively predict the next tokens. Since they are trained on vast amounts of text from the internet, LLMs can often be adapted to specific tasks, such as code generation. This is especially true for Python, which is the most widely used programming language. In these cases, a carefully designed prompt is typically sufficient to guide the model. However, while prompt based adaptation can be effective, fine-tuning these models on data from the target task usually results in significantly better performance.

The LLMs used in this work are text-only models, meaning they operate solely on textual input. However, some models from the same family are multimodal and can process both text and images. Providing the image associated with the query could potentially help the model generate more accurate or informed reasoning, especially in tasks involving visual understanding through code generation. Nonetheless, this work investigates the capabilities of language-only models, and all experiments are conducted under this constraint.

Among the LLMs used in this thesis, we have to distinguish between base and instruction-tuned models.

Base models are large neural networks initially trained through unsupervised or self-supervised learning methods, typically employing language modeling tasks such as next-token prediction or masked-token prediction.

In contrast, instruction-tuned models are derived by further fine-tuning base models on datasets explicitly composed of instruction–response pairs. This training enables the model to reliably understand, generalize, and follow instructions provided by users.

A critical aspect in using instruction-tuned models effectively involves inputing the instructions in the exact same format as the one used during the instruction-tuning phase. Typically, these models expect inputs organized around specific conversational roles, such as *system*, *user*, and *assistant*. To ensure inputs conform to this standardized format, chat templates are employed.

This work evaluates a selection of text-only LLMs that vary in size, training objectives, and specialization. Specifically, both base and instruction-tuned models are considered. This diversity enables a broad assessment of their ability to reason and generate code given natural language prompts. The models included in this work are described below.

LLaMA 3.1

LLaMA 3.1 is a family of LLMs introduced by Meta [8] and based on the LLaMA 3 architecture. They come in three sizes: 8B, 70B, and 405B parameters, each with base and instruction-tuned versions. However, in this work we focus on the 8B parameter models, with the base `Llama-3.1-8B`[1] and the instruction-tuned version `Llama-3.1-8B-Instruct`[2].

CodeLlama

CodeLlama is a family of LLMs specifically designed for code generation and understanding. It was introduced in [9] and developed by further pretraining the LLaMA 2 model on a large dataset composed of code and related natural language. It includes four versions ranging from 7B to 70B

[1] https://huggingface.co/meta-llama/Meta-Llama-3.1-8B
[2] https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct

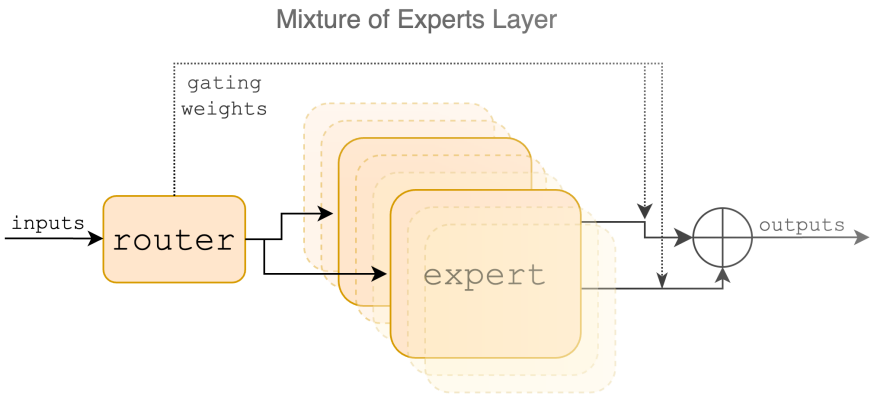

Figure 2.2: Mixture of Experts Layer. Each input vector is assigned to 2 of the 8 experts by a router. The layer's output is the weighted sum of the outputs of the two selected experts. In Mixtral, an expert is a standard feedforward block as in a vanilla transformer architecture. Image obtained from [10].

parameters with three variants each: the base model, an instruction-tuned version, and a Python-specialized version. However, we just focus on `CodeLlama-7B-It-hf`[3] and `CodeLlama-7B-hf`[4].

Mixtral

Mixtral 8x7B, released by Mistral AI [10], is a high-quality sparse mixture of experts (SMoE) model. MoE models use fewer parameters per inference, resulting in faster speeds than dense models of equivalent size. However, all parameters must be loaded into memory, causing high VRAM usage (Mixtral 8x7B needs VRAM similar to a 47B dense model). Inference computation is efficient since only selected experts (2 out of 8) and shared parameters are activated per token. The instructed `Mixtral-8x7B-Instruct-v0.1` [5] model is used in this work. In our experiments, running inference in 16-bit precision required two NVIDIA A100 GPU with 80 GB of memory. An overview of Mixtral's expert routing mechanism is shown in Figure 2.2.

Qwen2.5

Qwen2.5-Math is a specialized language model from the Qwen 2.5 family developed by Alibaba Cloud, optimized for solving mathematical and coding problems. The Qwen 2.5 series improves upon earlier versions with enhanced reasoning and coding abilities, as introduced in [11]. It is particularly suited for symbolic computation, math word problems, and reasoning queries. In this work, we use the base `Qwen2.5-Math-7B`[6] and the instruction-tuned version `Qwen2.5-Math-7B-It`[7].

[3] https://huggingface.co/codellama/CodeLlama-7b-Instruct-hf
[4] https://huggingface.co/codellama/CodeLlama-7b-hf
[5] https://huggingface.co//mistralai/Mixtral-8x7B-Instruct-v0.1
[6] https://huggingface.co/Qwen/Qwen2.5-Math-7B
[7] https://huggingface.co/Qwen/Qwen2.5-Math-7B-Instruct

However, since the instruction-tuned version was performing poorly in our experiments, we also used the `Qwen2.5-7B-It`[8] model.

DeepSeek-R1-Distill Models

DeepSeek-R1-Distill models are smaller LLMs derived from open-source base models and fine-tuned using a custom distillation approach introduced by DeepSeek [12]. Unlike traditional knowledge distillation, where a student model learns from a teacher's logits and ground-truth labels, this method distills reasoning capabilities by fine-tuning smaller models on a supervised fine-tuning (SFT) dataset generated by the larger DeepSeek-R1 model and an intermediate checkpoint of DeepSeek-V3. This strategy effectively transfers the reasoning behavior of larger models into smaller ones.

The distilled models are based on existing open-source architectures, with minor modifications to configuration and tokenizer settings. In this work, we use `DeepSeek-R1-Distill-Llama-8B`[9] and `DeepSeek-R1-Distill-Qwen-7B`[10].

When using these reasoning models, it is recommended to avoid adding a system prompt and to place all instructions within the user prompt. This occasionally made it difficult to identify and isolate the generated code from the rest of the model's output. Furthermore, to encourage structured reasoning, the prompt should end with the token "`<think>`".

2.2.2 Vision-Language Models

Vision-Language Models (VLMs) are designed to understand and generate text based on visual inputs. They combine the capabilities of LLMs with visual processing, enabling them to perform tasks that require both language and vision understanding. VLMs have shown remarkable performance in tasks such as image captioning, visual question answering, and image-text retrieval.

BLIP

BLIP-2 is a VLM introduced in *Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models* [3]. BLIP-2 improves upon its predecessor BLIP [13], which proposed a unified vision-language pretraining method with a multimodal mixture of encoder-decoder architecture.

BLIP-2 consists of three key components: a frozen vision encoder (ViT-G/14), a lightweight Querying Transformer (Q-Former) that extracts informative visual features, and a frozen or partially frozen LLM (Flan-T5 or OPT) that handles text generation and reasoning. This makes BLIP-2 a powerful model for various vision-language tasks, including image captioning, visual question answering, and image-text retrieval. See Figure 2.3 for an overview of the BLIP-2 architecture.

[8] https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
[9] https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B
[10] https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B

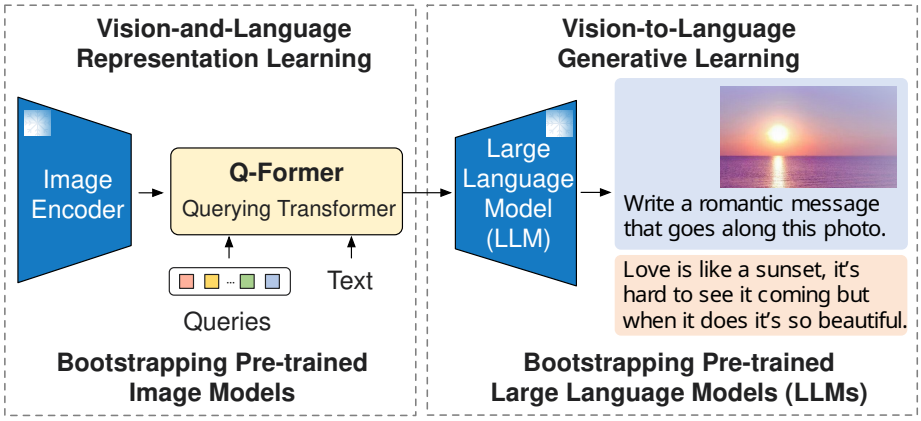

Figure 2.3: Overview of BLIP-2's framework. We pre-train a lightweight Querying Transformer following a two-stage strategy to bridge the modality gap. The first stage bootstraps vision-language representation learning from a frozen image encoder. The second stage bootstraps vision-to-language generative learning from a frozen LLM, which enables zero-shot instructed image-to-text generation. Image obtained from [3].

For our experiments, we used the BLIP-2 model with the Flan-T5-xl language model as the text decoder, called `Salesforce/blip2-flan-t5-xl`[11].

In the context of viperGPT, this module is called by the module `simple_query`, which handles basic queries over images that are not further decomposable.

X-VLM

Existing methods for learning vision-language alignments generally fall into two categories: fine-grained (object-centric) and coarse-grained (global) approaches. X-VLM is a vision-language model introduced in *Learning Cross-Modal Vision-Language Matching for Vision-Language Pretraining* [14]. It builds upon previous vision-language pretraining models by explicitly optimizing image-text alignment using multi-grained features. X-VLM demonstrates strong performance across a variety of vision-language tasks. In our work, we focus specifically on its capability for *Matching Prediction*, which determines whether a given pair of visual concept and textual description are semantically aligned.

Its architecture consists of an image encoder (I_{trans}), a text encoder (T_{trans}), and a cross-modal encoder (X_{trans}). All encoders are based on Transformer [15]. The cross-modal encoder fuses the vision features with the language features through cross-attention at each layer. See Figure 2.4 for an overview of the X-VLM architecture.

For our experiments, we employed the X-VLM model provided within the ViperGPT framework. Specifically the variant finetuned for retrieval tasks on the MSCOCO dataset, which includes `clip-vit-base-patch16` [16] as the vision encoder and `bert-base-uncased` [17] as the text

[11] https://huggingface.co/Salesforce/blip2-flan-t5-xl

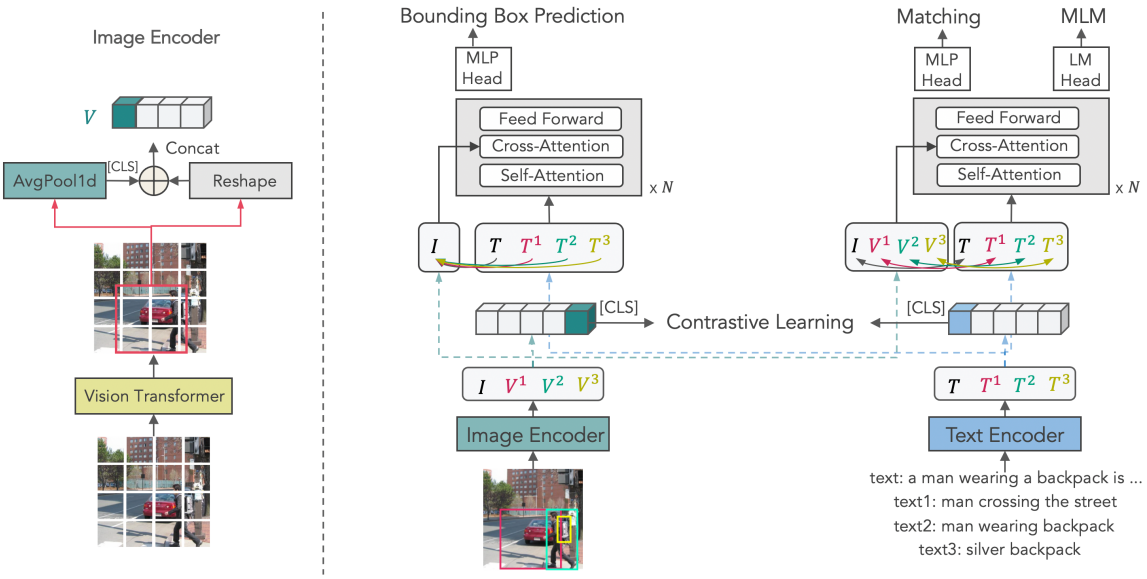

Figure 2.4: Pre-training model architecture and objectives of X-VLM. As shown on the left side, we extract features from the subset of patches from the vision transformer to represent images/regions/objects (I and V^{1-3}), which are then paired with corresponding text features (T and T^{1-3}) for contrastive learning, matching, and MLM. Meanwhile, the image (I) is paired with different textual descriptions (T and T^{1-3}) for bounding box prediction to locate visual concepts in the image.Image obtained from [14].

decoder. Within ViperGPT, X-VLM is accessed via the `best_text_match` module, which takes a single image and a list of noun phrases as input, and returns the phrase that best aligns semantically with the visual content.

It is especially useful for resolving references to specific objects or entities in the image, such as "the red car" or "a slice of cake," by identifying which candidate phrase is most semantically aligned with the image.

GLIP

GLIP (Grounded Language-Image Pre-training) is a VLM introduced in *GLIP: Grounded Language-Image Pre-training* [18]. Unlike traditional object detectors trained purely on visual data, GLIP is trained jointly on image-text pairs to unify object detection and phrase grounding. It is capable of detecting objects in images based on natural language descriptions, making it particularly effective for tasks that involve open-vocabulary object recognition.

GLIP is built upon the Grounded Pretraining paradigm, which leverages both vision-language datasets and standard object detection annotations. Its architecture consists of a visual backbone (ResNet or Swin Transformer), a language encoder (such as BERT), and a region-text matching module. This design allows the model to localize and identify objects directly from textual input,

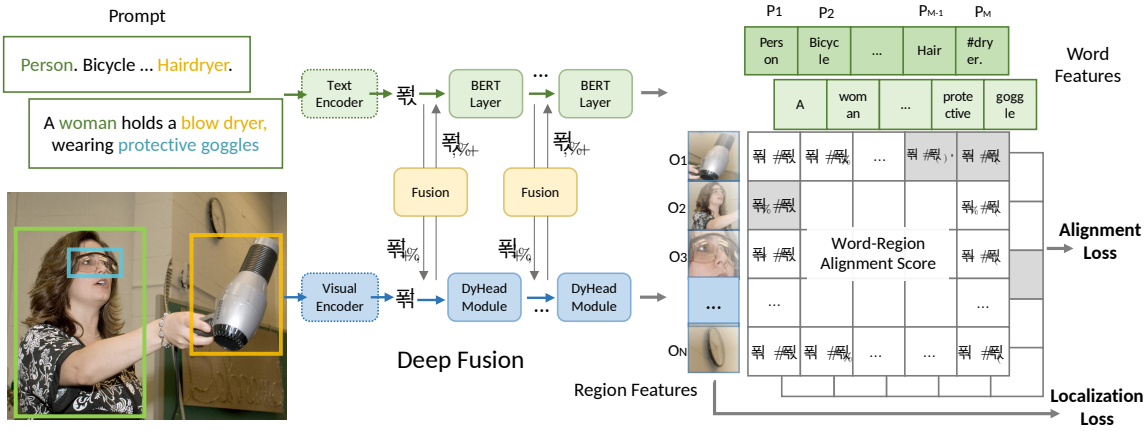

Figure 2.5: A unified framework for detection and grounding. Unlike a classical object detection model which predicts a categorical class for each detected object, we reformulate detection as a grounding task by aligning each region/box to phrases in a text prompt. GLIP jointly trains an image encoder and a language encoder to predict the correct pairings of regions and words. We further add the cross-modality deep fusion to early fuse information from two modalities and to learn a language-aware visual representation. Image obtained from [18].

even when those objects are not part of a fixed label set. See Figure 2.5 for an overview on the framework.

GLIP is available in five model variants; for our experiments, we used the GLIP-L version, which is based on the Swin-Large backbone and pretrained on the Visual Genome dataset [19]. It is worth noting that the GQA dataset leverages scene graph structures extracted from Visual Genome to construct its visual questions. This overlap in data sources may partially explain the observed drop in performance from the development set to the test set, as we will discuss in subsequent sections.

In our experiments, GLIP is used to support visual grounding in the ViperGPT framework. Specifically, we expose its functionality via two modules: `find` and `exists`.

These modules make GLIP particularly useful for object presence verification and spatial reasoning tasks within the ViperGPT pipeline. They allow the system to interface with fine-grained image regions based on open-ended natural language inputs.

2.3 Adaptation methods

Once a LLM completes its initial pre-training phase, it possesses a broad foundation of general knowledge and linguistic capabilities. However, to be genuinely useful for specific applications or to align with human preferences, this base model must undergo a crucial subsequent stage known as post-training or adaptation. This process refines the model's behavior, steering it towards

desired outputs. This section details the primary adaptation strategies employed: Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO). Furthermore, to address the significant computational demands of these methods, we incorporate Low-Rank Adaptation (LoRA), a parameter-efficient technique that makes the fine-tuning process substantially more manageable.

2.3.1 Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT) is a technique for adapting LLMs to specific downstream tasks or custom datasets. This process leverages a labeled dataset, where each instance consists of a prompt (input) and a desired completion (output), to further train the model. The primary goal of SFT is to refine the model's capabilities, making it more adept at generating relevant and accurate responses for the target application by learning task-specific patterns and knowledge.

The core mechanism of SFT involves minimizing a loss function, typically a cross-entropy loss, between the model's predicted token probabilities and the ground truth tokens of the desired response. Given a dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, where x_i is the prompt and y_i is the target completion, the SFT objective for a language model M parameterized by θ can be expressed as:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{(x,y)\in D}\sum_{j=1}^{|y|}\log P(y_j|y_{<j}, x; \theta)$$

Here, y_j is the j-th token of the target sequence y, and $y_{<j}$ represents the preceding ground-truth tokens. This strategy of using the actual ground-truth tokens from the target sequence as context for predicting subsequent tokens during training is known as teacher forcing. It helps stabilize training and guides the model effectively by providing correct inputs at each step of the sequence generation process. The model's parameters θ are updated iteratively using gradient-based optimization algorithms, such as Adam or SGD, by backpropagating the gradients of this loss function.

A key detail in the SFT process, particularly for instruction-following or dialogue generation, is the technique of masking the prompt tokens during the loss computation. This ensures that the model is only penalized for errors in generating the desired response, not for reproducing the input prompt. The loss is calculated exclusively on the tokens corresponding to the target completion. For instance, if an input sequence is structured as `[prompt_tokens][response_tokens]`, the loss is only computed over `[response_tokens]`. This focuses the learning on the generation aspect.

The use of special tokens is also prevalent in SFT to delineate different parts of the input and output sequences. Tokens such as `[SOS]` (start of sequence), `[EOS]` (end of sequence), `[SEP]` (separator), and `[PAD]` (padding) help the model understand the structure of the data. For example, an input sequence might be formatted as: `[SOS]<user_prompt>[SEP]<model_response>[EOS]`. The vocabulary of the model must include these tokens.

Furthermore, the choice of the original model for SFT can significantly impact the efficiency and effectiveness of the adaptation. It is often more beneficial to perform SFT on an instruction-

tuned model rather than directly on a raw base pretrained model. Instruction-tuned models have already learned to follow instructions and generate coherent, helpful responses in a general sense. Therefore, the adaptation required for a specific downstream task might be considerably lower. Additionally, these instruction-tuned models typically have already incorporated such special tokens and formatting conventions, streamlining the SFT process and potentially leading to better performance with less task-specific data. This is because the model has already developed a foundational understanding of how to interpret prompts and structure answers, reducing the burden on the SFT phase.

2.3.2 Direct Preference Optimization (DPO)

Direct Preference Optimization [20] was introduced as a more stable, performant, and computationally lightweight alternative to the dominant Proximal Policy Optimization (PPO) [21]. DPO cleverly sidesteps the need for explicit reward model fitting and the subsequent reinforcement learning phase. Instead, it introduces a novel parameterization of the reward model within the RLHF objective that allows the optimal policy to be extracted in closed form. This enables the standard RLHF problem to be solved using only a simple classification loss (see Figure 2.6).

The core idea of DPO is to directly optimize the language model policy to align with preferences, which are typically provided as pairs of *preferred* (y_w) and *rejected* (y_l) responses to a given prompt (x). Like traditional RLHF methods, DPO often relies on a theoretical preference model, such as the Bradley-Terry model, which states that human preferences can be modeled based on an underlying latent reward function. However, instead of using this model to train a separate reward function, DPO employs a change of variables to define the preference loss directly as a function of the policy itself. The resulting DPO objective seeks to increase the relative log probability of *preferred* responses compared to *rejected* ones. This is formulated as a binary cross-entropy objective:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

where π_θ is the policy being optimized, π_{ref} is a reference policy (the initial LLM), $\mathcal{D}$ is the dataset of preferences, β is a hyperparameter controlling the deviation from the reference policy, and σ is the logistic function. The term $\hat{r}_\theta(x, y) = \beta\log\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ represents the reward implicitly defined by the language model π_θ and the reference model π_{ref}. This formulation means the policy network effectively embodies both the language model and the implicit reward function. The gradient of this loss function intuitively increases the likelihood of *preferred* completions y_w and decreases the likelihood of dis*preferred* completions y_l, weighted by how incorrectly the implicit reward model currently orders the completions (scaled by β). The value of β directly affects the strength of the preference signal: increasing β sharpens the distinction between *preferred* and *rejected* completions, potentially accelerating learning but also risking instability or overfitting, while decreasing β softens the learning signal, resulting in more conservative updates that may stabilize training but slow convergence. This dynamic, per-example importance weighting is crucial for preventing model degeneration.

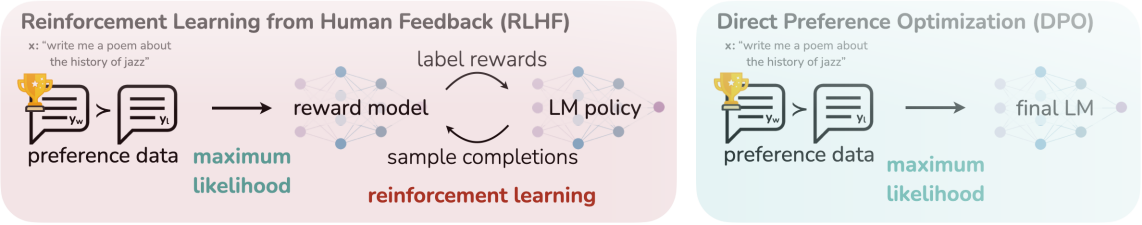

Figure 2.6: DPO optimizes for human preferences while avoiding reinforcement learning. Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an implicit reward model whose corresponding optimal policy can be extracted in closed form. Image obtained from [20].

The metrics used to monitor DPO training are the loss (explained above) and the rewards/accuracies. The rewards/accuracies metric serves as a primary indicator of how effectively the policy is learning the preference distribution. It is defined as the mean frequency with which the implicit reward of the *preferred* response (y_w) exceeds that of the *rejected* response (y_l).

Specifically, for each preference pair (x, y_w, y_l) in a given training batch, we evaluate the condition $\hat{r}_\theta(x, y_w) > \hat{r}_\theta(x, y_l)$. The rewards/accuracies metric is then calculated as the percentage of preference pairs for which the *preferred* response receives a higher implicit reward than the *rejected* one. For instance, an accuracy value of 0.85 indicates that the policy correctly ranks the *preferred* completion in 85% of the cases within that batch.

The DPO pipeline generally begins, optionally, with a base model fine-tuned via SFT on high-quality data for the task of interest; this model often serves as the reference policy π_{ref}. Subsequently, a dataset of preferences $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$ is collected, where completions y_w, y_l might be sampled from π_{ref} for each prompt x. Finally, the language model π_θ is optimized by minimizing the $\mathcal{L}_{\text{DPO}}$ with respect to its parameters.

2.3.3 Low-Rank Adaptation (LoRA)

Full fine-tuning of LLMs presents significant computational challenges. When training a model we do not just load the entire model weights, but we also need to store the gradients, optimizer steps, and forward activations.

Techniques like LoRA (Low-Rank Adaptation) [22] aim to make this process more feasible. LoRA freezes the pretrained model weights (W_0) and injects smaller, trainable rank decomposition matrices (A and B) into specific layers, typically within the Transformer architecture. The update to a weight matrix is thus represented as $\Delta W = BA$, where only A and B are optimized. These low-rank matrices A and B can be merged with the original weights W_0 during deployment ($W = W_0 + BA$), so when we say we are training a model, we are just training the LoRA adapters,

thus we only need to store A and B.

LoRA can decrease the number of trainable parameters by as much as 10000 times, potentially representing only 0.01% of the original model's parameters. For instance, with the 8B Llama model just 167M added parameters, out of 8B, were trained.

LoRA is extremely useful with DPO. For DPO setup one would typically need to load both the policy model (π_θ) and a reference model (π_{ref}) into VRAM, apart from the stuff aforementioned. However LoRA allows π_θ to be represented as $\pi_{\text{ref}} + \text{LoRA Adapter}$, saving the memory cost of loading an entire second model as π_θ is being updated with the LoRA Adapter.

However, LoRA is not without limitations. Since fewer parameters are updated, it might be less effective than full fine-tuning for tasks requiring very substantial changes to the base model's knowledge, where the low-rank constraint could be too restrictive. The choice of rank (r) and the specific layers to apply LoRA to are also hyperparameters that may require tuning for optimal performance.

2.4 Multimodal Datasets

Multimodal datasets are collections of data that incorporate information from multiple modalities. Instead of just focusing on pure text, multimodal datasets often include images, audio, video, or other modalities. These datasets are central to advancing research in tasks such as image captioning, visual question answering (VQA), speech-to-text, video understanding, and others.

Over the past decade, the VQA task has evolved to assess increasingly sophisticated capabilities in multimodal reasoning. While early benchmarks focused on general object recognition and scene understanding, recent datasets aim to challenge models with compositional reasoning, external knowledge requirements, and fine-grained visual grounding.

The **OK-VQA** dataset [23] introduces a challenging extension of the standard VQA task by requiring external commonsense or factual knowledge. Unlike traditional VQA datasets, where answers can typically be inferred directly from the image, OK-VQA contains questions whose correct answers rely on world knowledge beyond what is visually depicted. For instance, a question such as *"What country is this food from?"* may require knowledge of cultural cuisine or geography.

RefCOCO, along with its variants RefCOCO+ and RefCOCOg [24], focuses on referring expression comprehension. Given an image and a natural language phrase (e.g., *"the man in the red shirt"*), the task is to locate the corresponding object in the image. While not a question-answering dataset per se, RefCOCO plays an important role in evaluating visual grounding capabilities, which are fundamental to successful VQA performance.

While datasets like OK-VQA and RefCOCO target key aspects of multimodal understanding, we reduce the scope of this work to a single dataset that focuses on real-world visual reasoning. For this reason, the following analysis and experimentation will center exclusively on the GQA benchmark, as it provides a comprehensive framework for evaluating complex compositional

questions.

2.4.1 GQA: A Benchmark for Real-World Visual Reasoning

The GQA dataset [1] was introduced as a response to the limitations identified in earlier visual question answering benchmarks such as VQA v1 and v2. While those datasets provided an essential foundation for research in multimodal learning, they often suffered from strong language priors, limited diversity in reasoning types, and a tendency to allow models to exploit dataset biases. GQA was specifically designed to address these shortcomings by focusing on real-world visual reasoning and compositional question answering.

The GQA dataset is built on detailed scene graphs from Visual Genome, enabling the generation of over 22 million questions. To enhance robustness and reduce bias, a balanced subset of 1.7 million questions was created through controlled downsampling, ensuring diversity and uniformity across question types and answers. Questions test a range of reasoning skills, such as spatial understanding, logical inference, and commonsense reasoning.

In addition to standard accuracy, GQA introduces several diagnostic metrics to evaluate different aspects of model performance. *Consistency* measures whether a model provides logically coherent answers across semantically related questions. *Validity* checks whether the output falls within an acceptable range of plausible responses. *Plausibility* assesses the contextual suitability of an answer given the question structure. *Grounding* evaluates whether the answer is supported by visual evidence from the image, and *distribution* analyzes how closely the model's answer distribution matches that of the ground truth. These metrics provide a multi-faceted evaluation framework that goes beyond surface-level correctness and encourages deeper visual understanding. However, since our model was not end-to-end, using these metrics was quite complicated, and only accuracy was used.

Accuracy is computed as the proportion of predictions that exactly match the ground truth after applying a post-processing step to handle type and formatting inconsistencies. This post-processing includes converting booleans to "yes"/"no", extracting values from tensors or lists, handling None values, casting numerical predictions to strings, and lowercasing and trimming whitespace.

In our work, we focus exclusively on the balanced version of the dataset, which is organized into three main partitions:

(a) The training set, distributed as `train_balanced_questions.json`, is used for the generation of synthetic data. It contains 943000 instances, but only the first 12600 are used to create the dataset due to time and computational constraints.

(b) The validation set, `val_balanced_questions.json`, is used to monitor model learning and tune hyperparameters. It contains 132062 instances, but only the first 1000 are used to create a small validation set to enable relatively fast evaluation of LLM generation performance.

Figure 2.7: Compositional image question answering on GQA. Image obtained from [1].

(c) The testdev set, referred to as `testdev_balanced_questions.json`, serves as our final evaluation split and contains 12578 questions. Although evaluating all models on this large number of instances was time-consuming, it was necessary to ensure comparability with state-of-the-art results.

In Figure 2.7, we present a few examples of the types of questions included in the GQA dataset. These examples illustrate the compositional nature of the questions, which often require multiple reasoning steps and a deep understanding of the visual content to answer correctly.

For instance, the first visual query asks about two properties of an item, in this case whether a pancake is round and brown. To answer this question, humans would typically first need to identify the pancake in the image, then determine its shape and color. This requires both visual recognition and reasoning about the properties of the identified object and are performed sequentially. This reasoning is perfectly shown in the code, where the first step is to find the pancake in the image, for that the `find` module is used for fine-grained comprehension. Then, the two properties are checked using the `verify_property` module, which uses the X-VLM model to check if the color and shape match the expected values. With that info, an answer is generated.

The second visual query in Figure 2.7 demonstrates a more complex spatial reasoning task. The question asks whether there are water bottles to the right of a wooden bookcase. To answer this, the model must first identify the relevant objects in the image: the bookcase and the water bottles. It must then reason about their spatial relationship based on their positions within the visual scene.

The generated code follows a step-by-step reasoning process. It first locates the bookcase using the `find` module, which extracts image patches corresponding to regions identified as bookcases. As there may be more than one, the model then verifies whether the detected bookcase is made of wood by calling the `verify_property` module. This step ensures that only bookcases with the specified material are considered, and relies on the X-VLM model to confirm the material attribute.

If a wooden bookcase is found, the model continues by locating water bottles in the image using another `find` operation. It then checks whether any of the water bottles are positioned to the right of the bookcase by comparing their horizontal center coordinates. Specifically, it verifies whether the horizontal center of a water bottle is greater than that of the bookcase. If this condition holds, the model returns "yes"; otherwise, it returns "no."

This example illustrates how the GQA dataset requires combining attribute verification with spatial reasoning in order to correctly answer visual queries.

CHAPTER 3

Dataset creation for GQA

The first step in this work was to collect a large amount of data to enable the training of LLMs for visual reasoning tasks. Since ViperGPT follows a few-shot learning approach, where they just adapt the model via prompt engineering, we cannot directly adapt the model weights to improve its code reasoning capabilities. Therefore, the first step is to create a dataset that captures diverse forms of code-based reasoning grounded on visual questions, and then use this dataset to train the LLMs. We consider two types of datasets: one designed for DPO, which requires a set of preferences between generated codes for each instance, and another for SFT, which requires ground-truth code sequences, which are codes that generate the correct answer to the visual question.

3.1 Datasets construction process

To generate the dataset, we used six different language models, which are described in Section 2.2.1. For all models, we adopted the recommended generation configuration specified in Huggingface's model repository, with one key modification: the temperature parameter was always set to 1. This choice was made to encourage diversity in reasoning strategies. Many of the questions in GQA are structurally similar, and since the models do not have access to the image itself during generation, deterministic sampling often results in identical or near-identical code. Although using a temperature of 1 slightly reduces the per-model accuracy, it provides more diverse reasoning paths, which is beneficial for our goal of modeling reasoning diversity.

The generation was performed on the balanced training split of the GQA dataset, specifically on the first 12600 samples (see Section 2.4.1 for details on the GQA dataset splits).

Once the code was generated for each question, we executed the programs to obtain answers to the visual questions and evaluate the code quality. The evaluation results are shown in Table 3.1.

LLM	Code Error	Runtime Error	Semantic/Inference Error	Correct
Llama3.1-8B-It	155	199	6018	6228
CodeLlama-7B-hf-It	193	997	6476	4934
Mixtral-8x7B-It	295	1646	5802	4857
DeepSeek-Llama8B	1152	1007	5658	4783
Qwen2.5-Math-7B	2089	1976	5057	3478
DeepSeek-Qwen7B	3018	2026	4480	3076

Table 3.1: Performance of each LLM on the balanced training-question set, over 12600 instances. GLIP-Large and BLIP2-Flan-T5-XL (32-bit) were used as VLMs.

Based on the execution results, we categorized each outcome into one of the following four classes:

Correct answer: The generated code correctly answers the visual question after execution, indicating that the generated code demonstrates effective reasoning. Check Section 2.4.1 for more details.

Semantic or inference error: The code fails to produce the correct answer. This can be caused by an inference error, where the code is logically sound, but the VLMs invoked during execution make incorrect inferences; or a semantic error, where the code itself lacks appropriate reasoning logic, leading to an incorrect answer. Distinguishing between these two types of errors would require a manual analysis of the code to determine whether the reasoning is valid or not. Since there is no automatic way to make this distinction reliably, both types of errors are grouped under the same category.

Runtime error: The code runs but fails due to runtime issues. This can be caused by errors that occur during execution, such as incorrect function usage or invalid data handling; or by invoking a VLM that fails to process the input correctly (for example, returning empty lists due to missing object detections). Although this latter case could be considered an inference error where the code is still semantically correct, for the sake of simplicity, we include it under the runtime error category.

Syntax error: The generated code contains syntax errors that prevent it from being parsed or executed.

It is worth noting that certain edge cases remain unaccounted for in this classification. For example, a semantically incorrect code may still yield a correct result if a VLM happens to fail in a way that aligns with the expected output. However, these cases are not systematically addressed in our analysis, as their occurrence is considered highly unlikely. In fact, no such instances were observed during our qualitative analysis.

Pattern	Compilation Error	Runtime Error	Semantic or Inference Error	Correct	# instances
A				✓	443
B			✓	✓	2209
C		✓	✓	✓	1973
D	✓	✓	✓	✓	1198
E	✓		✓	✓	1820
F		✓		✓	447
G	✓	✓		✓	375
H	✓			✓	409
I			✓		793
J		✓	✓		1078
K	✓	✓	✓		846
L	✓		✓		1008
M		✓			0
N	✓	✓			1
O	✓				0
				Total	12600

Table 3.2: Distribution of the 12600 visual question instances according to the classifications of the 6 generated codes per instance.

We have developed a system that assigns each evaluated program to one of the previously defined categories. Because these categories form a strict hierarchy, we can quickly compare the quality of the reasoning shown by different pieces of code.

Figure 3.1: Ideal preference hierarchy with five distinct reasoning-quality levels.

In the ideal scenario, the first two categories still exhibit satisfactory reasoning, while the remaining ones contain increasingly undesirable code (see Figure 3.1).

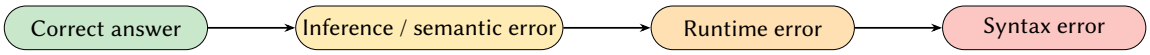

Figure 3.2: Practical preference hierarchy used by the automatic classifier.

Since an automated classifier cannot reliably distinguish inference errors from semantic errors, these two are merged in practice, even though this may reduce training efficiency, as the model

could be discouraged from generating code that is actually semantically correct (see Figure 3.2).

This strict ordering lets us build a preference system, we can take any two candidate codes for the same visual question and decide which one shows better reasoning. The procedure resembles Reinforcement Learning from Human Feedback, except that here the labelling is automatic rather than manual. From those pairwise decisions we can construct a dataset of preferences, selecting the pairs through a heuristic that focuses on the most informative comparisons (those expected to provide the strongest learning signal to the model).

Table 3.2 provides highly relevant information about the synthetic codes that were generated. The ticks indicate that at least one of the six models produced a code that falls into the corresponding category; for example, in category H at least one model generated a correct code and at least one model produced a code with a compilation error, while none of the generated codes belong to any other category.

Because there are four possible error categories, the table contains $2^4 - 1 = 15$ distinct patterns. These patterns reveal which instances deliver the strongest learning signal. Instances belonging to pattern O, M, I are the least informative because all of them show bad reasoning and none of them can be used for preference comparisons. Patterns N also exhibits bad reasoning, but with this we could teach the model to avoid code that fails to compile.

Patterns J, K and L found in yellow in Table 3.2, provide a stronger learning signal: they can help the model learn not to generate code with runtime or compilation errors, although correct reasoning is not explicitly reinforced (it might emerge implicitly if a model produced a correct code while the VLMs caused the answer to be incorrect, but this is uncertain).

Patterns B, C, D, E, F, G and H, found in light green in Table 3.2, offer the strongest learning signal. Each of these includes at least one correct code and at least one code of lower hierarchy, enabling us both to encourage correct output and to discourage incorrect solutions. Pattern A, found in dark green in Table 3.2, contains just correct codes and no incorrect alternatives, so it cannot be used for preference comparisons but is suitable for supervised-fine-tuning (SFT).

Across the dataset, 8874 out of 12600 visual questions have at least one correct answer, implying an upper-bound accuracy of roughly $8874/12600 \approx 0.7$ if the knowledge of all six models could be consolidated into a single model. An open question is whether this upper bound can be reached or surpassed if the model successfully generalises the knowledge transferred during training.

3.1.1 Preference dataset

To build the preference dataset, we retained only the 8431 instances that fall into patterns B–H.

Two principal preference datasets were generated. The first approach randomly selects two codes for each instance under the condition that at least one is correct and the other is incorrect (belongs to a lower-hierarchy category). This dataset, referred to as single-pair preference dataset,

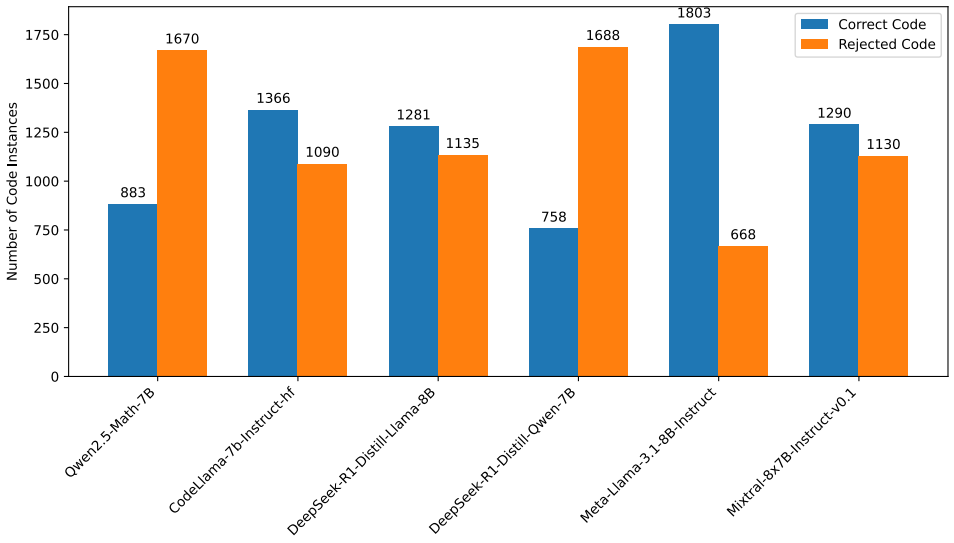

Figure 3.3: Distribution of preference pairs in the training partition of the single-pair DPO dataset.

contains 8431 pairwise comparisons, each formed by a *preferred* and a *rejected* code as defined in the literature.

The corpus is divided into training and development splits: 1000 instances are reserved for development and the remainder for training. This division was performed at random and will be used as the validation set for every training run, regardless of which dataset is employed. Consequently, it affects the construction of the other datasets. The visual questions that belong to this validation set must be excluded so that no contamination occurs in the additional training datasets.

Figure 3.3 shows the distribution of models producing the *preferred* and *rejected* codes in the training partition of the single-pair DPO dataset. As expected, Llama3.1-8B-It, being the best performing model, exhibits the highest number of *preferred* codes and the fewest *rejected* codes. Conversely, Qwen2.5-Math-7B and DeepSeek-R1-Distill-Qwen-7B have the largest numbers of *rejected* codes and the fewest *preferred* codes. The distribution of the models in the development follows a very similar shape, as it was expected since the development set was randomly selected from the training set. See Figure C.1 in Appendix C for details.

The second approach to constructing a preference dataset, in this case referred to as the all-pair preference dataset, involves selecting all code pairs in which one code is correct and the other is incorrect (belongs to a lower-hierarchy category).

With this approach we can expect a much larger dataset. Take into account that per instance we have 5 to 9 possible combinations to select the pairs, depending on how the codes are distributed

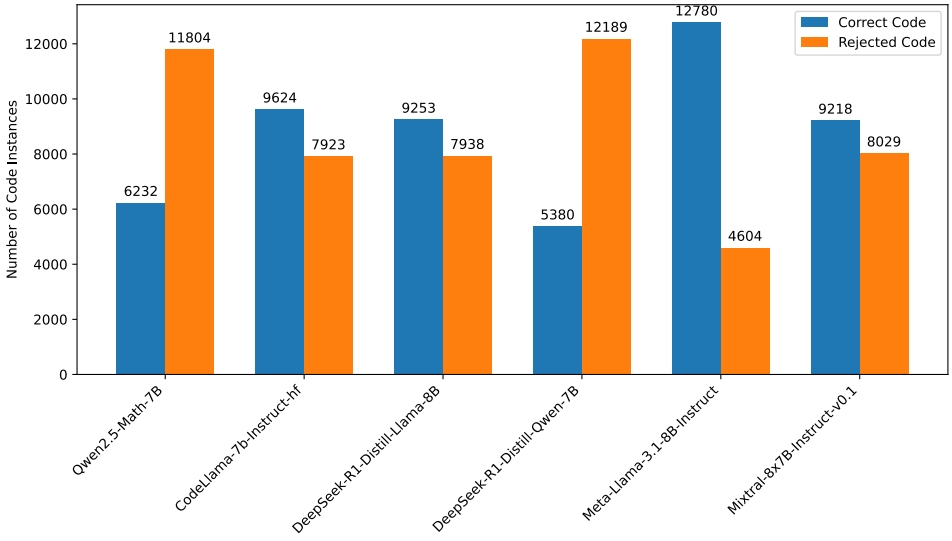

Figure 3.4: Distribution of *preferred* and *rejected* codes per model in the every-pair dataset.

among the different classifications. The worst case scenario would be to have just one correct code, then we will have to choose between the 5 remaining codes as *rejected* ones (and viceversa if we had only one incorrect code). In the best case scenario, the codes would be evenly distributed and we would have 9 possible combinations to choose the pairs.

After discarding the 1000 visual questions that are part of the development set, which amount to 7112 code pairs, the final dataset consists of 52487 training pairs out of the original 59599. This makes the second dataset considerably larger than the first, although many of the codes in this set tend to be more similar to each other.

As shown in Figures 3.3 and 3.4, with both variants of the dataset, Llama3.1-8B-It stands out with the highest number of *preferred* codes and the lowest number of *rejected* ones. This observation is relevant as it is expected to influence the model's training performance.

3.1.2 SFT dataset

To build the Supervised Fine-Tuning (SFT) dataset, we retain the 8874 instances that fall into patterns A–H. We know that these visual questions possess at least one correct code, so we can simply select a random code from each instance and obtain a dataset of 8874 question answer pairs. However, 1000 instances must be reserved to generate a validation dataset. We decided to filter the same set of queries used on the development set for the preference dataset. This strategy also enables the use of the preference development set for evaluating the model's training progress during the supervised fine-tuning.

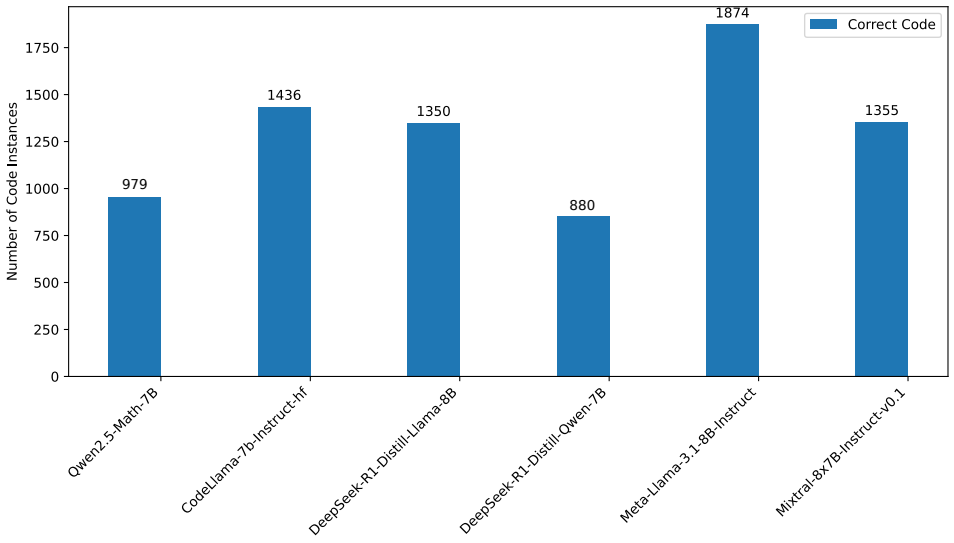

Figure 3.5: Distribution of generated codes per model in the SFT dataset for the train partition.

Furthermore, this could be particularly important if we wanted to make a direct comparison between the training in both datasets, or in scenarios involving using both preference and SFT datasets in a two phase fine-tuning process, where the model is first trained on the SFT dataset and then on the preference dataset, ensuring that no data contamination occurs in the validation set.

As shown in Figure 3.5, the distribution of models selected to populate the SFT dataset is similar to the one of the preference dataset. To see the distribution of the models for the development set refer to Figure C.2 in Appendix C.

3.2 Dataset Statistics

As shown in Figure 3.6, the main pattern we observe is that *preferred* codes tend to be shorter than *rejected* ones, except in the case of very short codes, which are more frequently rejected. These very short generations often occur when the model fails to perform composite reasoning and instead issues a single `simple_query` call using BLIP, this is analyzed in Section 5.6.

Excluding this case, we can conclude that, in most instances, shorter code with fewer reasoning steps is preferred. One possible explanation is that longer code typically involves more calls to VLMs, increasing the likelihood of failure in at least one of those calls.

The distribution of code lengths for the every-pair DPO dataset is very similar to the one of the single-pair DPO dataset. It can be seen in Figure C.3 in the Appendix C.

Further analysis was conducted on the composition of the training data to verify that the

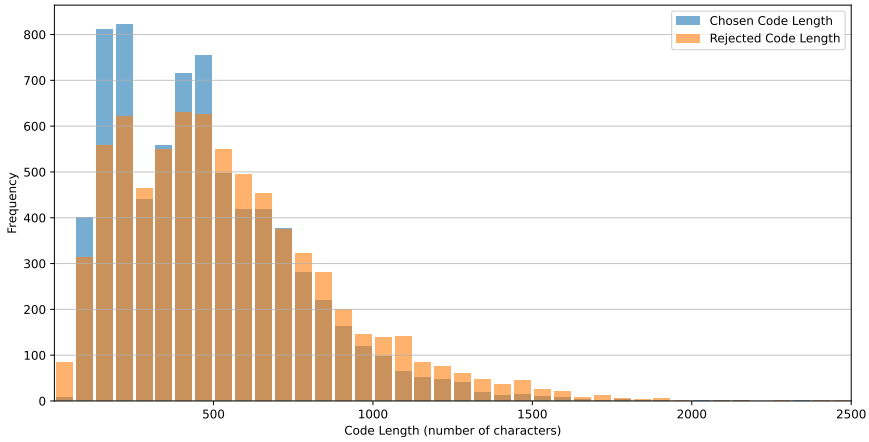

Figure 3.6: Generated code length distribution comparison between chosen and *rejected* samples in single-pair approach

instance selection process for our SFT dataset did not significantly alter the underlying data distributions from the original GQA subset. The Figure 3.7 shows the distribution of gold truth answers for the 8874 instances in our SFT dataset against the distribution for the larger 12600 instance subset from the GQA training split. Since the SFT dataset is almost entirely derived from the selected examples for the DPO dataset, this analysis is also relevant for understanding the

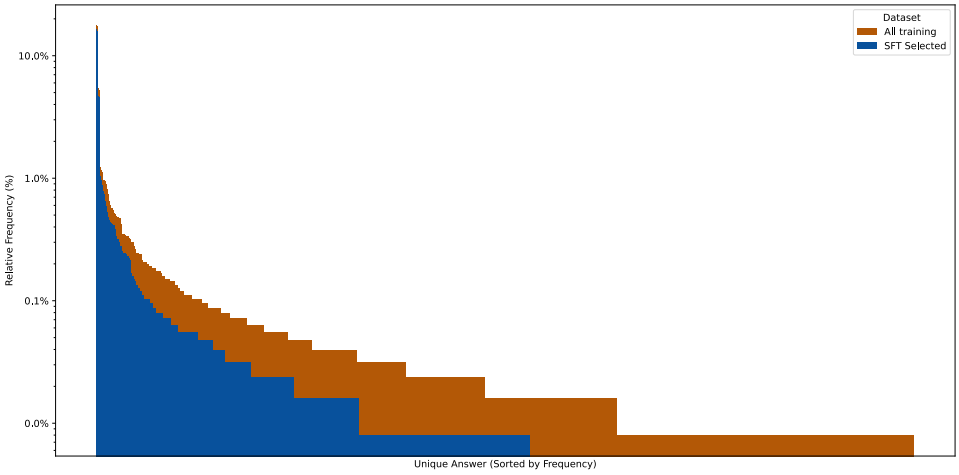

Figure 3.7: Distribution of the gold truth answers from the subset of 12600 instances selected from the train split of GQA compared to the subset of 8874 instances used for the SFT dataset.

DPO dataset. As expected, because the SFT set is a subset, its frequency counts are consistently lower than those of the full GQA subset.

The original GQA subset (All training) exhibits a classic long-tail distribution, containing a wide variety of answers that appear infrequently. In contrast, the SFT dataset's tail (SFT Selected) is substantially truncated. This suggests that the selection criteria, which was based on the correctness of the answers, filtered out many of the rarer, long-tail answers. A plausible explanation is that these unique answers correspond to more challenging questions where the models failed to produce correct reasoning paths, leading to their exclusion from the final SFT dataset and a potential loss of valuable, diverse information.

Figure 3.8 compares the distribution of query lengths, measured in characters, between the original subset of 12600 GQA instances and the 8874 instances chosen for the SFT dataset. Both datasets exhibit a peak frequency for queries between approximately 40 and 80 characters. The high degree of overlap indicates that the selection methodology for the SFT dataset did not introduce any discernible bias with respect to the length of the questions. We can, therefore, consider the SFT dataset to be a representative sample of the original GQA subset in terms of query length. The frequency is consistently lower in the SFT dataset, which is expected because it is a smaller subset of the GQA training set.

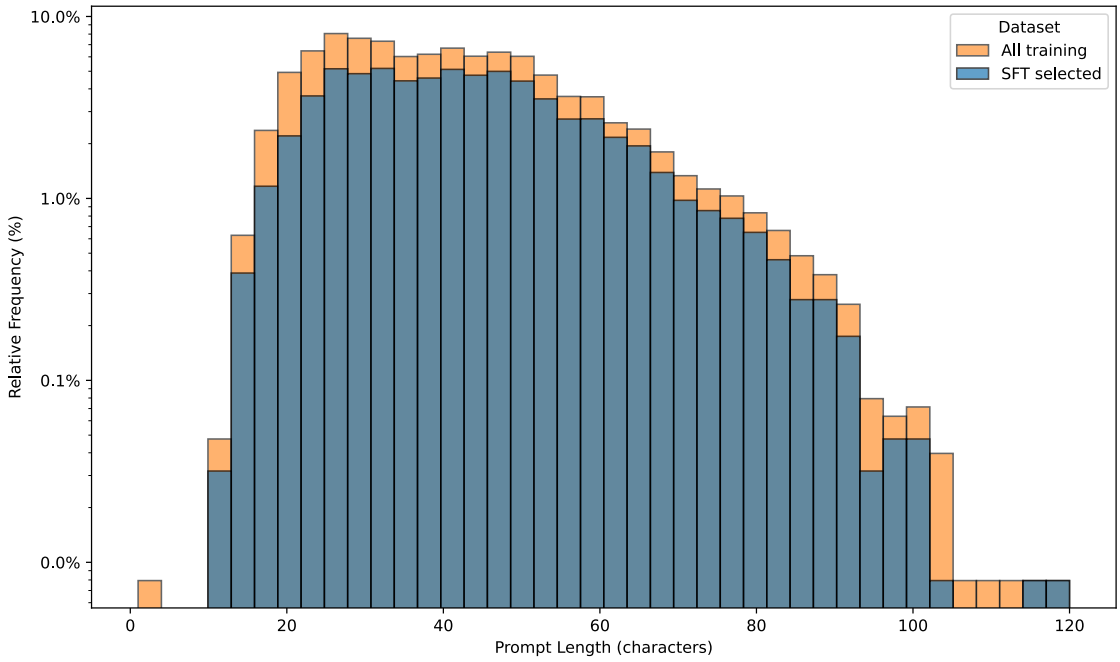

Figure 3.8: Distribution of the queries length from the subset of 12600 instances selected from the train split of GQA compared to the subset of 8874 instances used for the SFT dataset.

CHAPTER 4

Experiments

4.1 Experimental settings

To fine-tune the models two independent approaches have been followed. The first one is Direct Preference Optimization (DPO). The second one is Supervised Fine-Tuning (SFT).

Both approaches are used to improve the model's code generation capabilities, somehow transferring every model's knowledge to a single model, which can be seen as a wisdom of the crowd approach.

All training sessions were conducted using LoRA (explained in Section 2.3.3) to reduce computational and memory costs. Although full fine-tuning might have yielded better results, this approach was outside the scope of the thesis. The LoRA hyperparameters are detailed in Appendix E.

All open-weight models were trained and evaluated on a computing cluster equipped with an NVIDIA A100 80GB GPU. The complete set of training and evaluation runs is estimated to have required approximately 250 GPU hours, *not including* the time spent on discarded initial training runs, hyperparameter search, prompt experimentation, model changes, or training on alternative datasets.

4.1.1 DPO settings

To begin fine-tuning Llama3.1-8B, CodeLlama-7B-hf and Qwen2.5-Math-7B, several training tests were initially performed on the single-pair preference dataset. From these initial training runs, *loss* and *reward/accuracy* curves were obtained for the train and development sets (see Section 2.3.2 for details on the metrics). What really interested us here was analyzing these curves on the preferences development set to see how the model was generalizing to unseen data.

LLM (Fine-Tuned)	Dataset	Hyperparameters
Qwen2.5-Math-7B	DPO Every-pair	LR: 5e-5 BS: 16 β: 0.1 Scheduler: cosine Warmup: 0.1 Epochs: 2
CodeLlama-7B-hf	DPO Every-pair	LR: 5e-5 BS: 16 β: 0.1 Scheduler: cosine Warmup: 0.1 Epochs: 2
Llama-3.1-8B	DPO Every-pair	LR: 4e-5 BS: 32 β: 0.02 Scheduler: cosine Warmup: 0.2 Epochs: 2

Table 4.1: Hyperparameters used for each of the LLMs fine-tuned on the DPO dataset. Early stopping was applied based on the development loss.

It is important to note that these metrics only tell us if the model is learning to differentiate between good and bad code, but they do not tell us if the model is capable of generating correct code. To determine this, we used the validation set from GQA (see Section 2.4.1) to generate code with the trained model (using the LoRA adapter), and then the code was evaluated as it is usually done. From this, we can obtain a real comparison of how much the reasoning of the trained model has improved compared to the baseline models.

Having trained and evaluated the generation of several models, we measure the correlation between *reward/accuracy* and *loss* metrics, and the generation performance of the trained model. This analysis revealed a proportional correlation between them; that is, the higher the reward accuracy, the better the code generation, and the lower the loss, the better the code generation. This was not always true for different models and different training runs, but it was for the different checkpoints of the same training run. This finding was crucial for establishing the checkpoint saving strategy, which was based on the loss, and for performing early stopping.

Next, a hyperparameter search was conducted, beginning with the adjustment of the learning

rate. An attempt was made to increase the learning rate as much as possible while ensuring training stability. Batch sizes of 16 and 32 were tested, with no significant difference observed between them.

For the beta parameter in DPO, values between 0.01 and 0.2 were explored, but these variations did not yield significant differences in the outcomes (see Section 2.3.2). A cosine learning rate scheduler was utilized, with warmup proportions of 0.1 and 0.2 being tested.

We also compared the performance of the single-pair and all-pair preference datasets. The all-pair preference dataset yielded slightly better results, and was therefore selected for the final experiments (see Section 5.1 for further analysis).

The results of the hyperparameter search are shown in Table 4.1 and these are the hyperparameters used for the final training runs.

4.1.2 SFT settings

The development process followed for SFT was very similar to that of DPO. However, a key difference was the decision to use the instruction-tuned versions of the models: Llama-3.1-8B-It, CodeLlama-7B-hf-It, and Qwen2.5-7B-It, which are described in Section 2.2). Our focus was on maximizing model improvement, and we hypothesized that instruction-tuned models would adapt more quickly to the downstream task. Furthermore, this approach allowed us to use the specific instruction and formatting tokens with which these models had been pretrained, avoiding the need to define new ones.

A crucial step in our SFT methodology was the construction of a Custom DataCollator. This was essential for implementing the strategy of training on completions only, which means that the loss is computed just on the target completion part of the sequence, not on the whole input prompt. If this is not carefully managed, the loss function would be computed on the prompt tokens as well, which is undesirable. The Custom DataCollator was therefore designed to modify the labels such that tokens corresponding to the input prompt were masked (setting their labels to -100), ensuring that the loss was calculated exclusively on the model's generated completion.

To analyze the quality of the SFT training, the training loss was monitored continuously. Additionally, every 200 training steps, the loss was evaluated on the development reserved partition of 1000 instances.

Upon initiating the first SFT training runs, it was observed that all training and development loss curves exhibited very similar patterns, regardless of the hyperparameters employed. Initially, the loss decreased very rapidly, indicating that the model was effectively learning the downstream task. However, after approximately 100 training steps (which corresponds to 3200 instances, given a batch size of 32, representing nearly half of our training dataset), the training appeared to converge, as the loss values maintained the same on both the training and development curves. Based on these observations, further experiments were conducted with significantly higher learning rates, but this made the training unstable, leading to no positive results.

LLM (Fine-Tuned)	Dataset	Hyperparameters
Qwen2.5-7B-It	SFT Dataset	LR: 1e-4 BS: 32 Scheduler: cosine Warmup: 0.2 Epochs: 4
CodeLlama-7B-hf-It	SFT Dataset	LR: 1e-4 BS: 32 Scheduler: cosine Warmup: 0.2 Epochs: 4
Llama-3.1-8B-It	SFT Dataset	LR: 4e-5 BS: 32 Scheduler: cosine Warmup: 0.2 Epochs: 4

Table 4.2: Hyperparameters used for each of the LLMs fine-tuned on the SFT dataset. Early stopping was applied based on the development loss.

Despite numerous trials and modifications to the prompting structure, we were unable to get the Math version of Qwen (Qwen2.5-math-7B-It) to work correctly. It consistently generated output containing Chinese characters in the generated code. Therefore, we decided to discard this variant and continue using the standard version of Qwen.

However, even the standard Qwen2.5-7B-It model generated outputs that were problematic: many generations contained infinite loops or an excessive number of calls to vision modules. These issues hindered the evaluation process, as the generated code could not be reliably executed or interpreted. To mitigate this, we applied a second post-processing step to completely eliminate such codes from evaluation, since they were not desired reasoning outputs. Interestingly, as we will analyze later, fine-tuning appears to help this model implicitly eliminate the generation of such infinite code patterns.

The results of the hyperparameter search are shown in Table 4.2 and these are the hyperparameters used for the final training runs.

LLM	Syntax Error ↓	Runtime Error ↓	Semantic / Inference Error ↓	Correct ↑
Not Fine-Tuned (Baseline)				
ViperGPT	-	-	-	48.10
Qwen2.5-Math-7B	0.78	7.56	52.33	39.33
CodeLlama-7B-hf	0.78	0.60	55.61	43.02
Llama-3.1-8B	1.19	0.39	53.29	45.13
Fine-Tuned Models				
Qwen2.5-Math-7B	0.05	8.79	49.62	41.55 (+2.22)
CodeLlama-7B-hf	0.19	0.17	53.10	46.53 (+3.51)
Llama-3.1-8B	0.13	0.10	51.98	47.80 (+2.67)

Table 4.3: Comparison of zero-shot (including ViperGPT) and DPO fine-tuned LLMs on the GQA testdev partition. To evaluate our models GLIP-Large and BLIP2-Flan-T5-XL were used.

4.2 Results

Once the training sessions were completed, code was generated with validation set questions using the newly fine-tuned models, and this code was then evaluated. The best performing models were selected and subsequently evaluated on testdev split.

4.2.1 DPO Results

The DPO fine-tuned models achieved the results shown in Table 4.3 on the *testdev* set. The results show that the DPO fine-tuned models achieved a notable improvement in the code generation task compared to their respective baselines. All models demonstrated at least a 2% increase in the proportion of correct answers, with CodeLlama-7B-hf showing the largest gain of 3.51%.

Besides the correct answers, it can be seen that syntax errors have been reduced in all models, with the Qwen2.5-Math-7B model showing the most significant reduction from 0.78% to 0.05%. This indicates that the DPO fine-tuning process has effectively improved the syntax correctness of the generated code. This was expected because we were fully certain that the codes presenting a syntax error in the dataset were unpreferred.

The runtime errors have also been reduced in the CodeLlama-7B-hf and Llama-3.1-8B models, with the Qwen2.5-Math-7B model showing a slight increase in runtime errors.

The semantic errors were also reduced in all models, with the CodeLlama-7B-hf model showing the most significant reduction from 55.61% to 53.1%. This was the most complicated error to reduce, as it is not always clear that the generated code is semantically correct, it can be that the dis*preferred* codes were semantically correct, but the VLMs failed their evaluation.

LLM	Syntax Error ↓	Runtime Error ↓	Semantic / Inference Error ↓	Correct ↑
Not Fine-Tuned (Baseline)				
ViperGPT	-	-	-	48.10
Qwen2.5-7B-It	2.71	3.12	52.72	41.45
CodeLlama-7B-It-hf	0.06	1.14	54.79	44.01
Llama-3.1-8B-It	0.12	0.89	54.45	44.54
Fine-Tuned Models				
Qwen2.5-7B-It	0.01	0.06	53.67	46.27 (+4.82)
CodeLlama-7B-It-hf	0.02	0.05	53.03	46.90 (+2.89)
Llama-3.1-8B-It	0.03	0.14	52.92	46.92 (+2.38)

Table 4.4: Comparison of zero-shot (including ViperGPT) and supervised fine-tuned LLMs on the GQA testdev partition. To evaluate our models GLIP-Large and BLIP2-Flan-T5-XL were used.

4.2.2 SFT Results

The models trained with SFT achieved the results shown in Table 4.4 on the *testdev* set. The results indicate that SFT leads to improvements across all error categories: syntax errors, runtime errors, and semantic errors were consistently reduced in the fine-tuned models compared to their zero-shot counterparts.

The most notable improvement came with Qwen2.5-7B-It for which syntax errors decreased from 2.71% to 0.01% and runtime errors from 3.12% to 0.06%. In the case of CodeLlama-7B-It-hf, syntax errors decreased from 0.06% to 0.02%, and runtime errors dropped from 1.14% to 0.05%. Similar improvements were observed for Llama3.1-8B-It, which also exhibited substantial reductions in both syntax and runtime errors.

In terms of correct generations, all fine-tuned models achieved a higher percentage of correct answers. Specifically, Qwen2.5-7B-It improved from 41.45% to 46.27% (+4.82), CodeLlama-7B-It-hf improved from 44.01% to 46.90% (+2.89), and Llama-3.1-8B-It improved from 44.54% to 46.92% (+2.38). These results indicate that SFT effectively enhances the models' ability to generate correct code, particularly in terms of reasoning and logic.

4.2.3 DPO and SFT differences

When comparing the results of DPO and SFT, we observe that both methods lead to improvements in the models' performance on the GQA testdev set. However, there are notable differences in the nature of these improvements.

If we consider performance on the validation set (see Tables D.1 and D.2), SFT consistently yielded better results across all models. Nevertheless, on the *testdev* split, DPO results were

relatively stronger in comparison to SFT. The most notable case is the Llama-3.1-8B model, which achieved the best overall performance. This model reached 47.80% accuracy under DPO fine-tuning, compared to 46.92% with SFT.

The reason for this remains unclear. It seems that DPO training enabled the models to generalize better to the testdev data. In principle, the validation set already evaluates generalization, since it contains data unseen during training. However, our results suggest that the testdev split follows a different distribution than validation, and that DPO fine-tuning yielded more robust models under this shift, at least in the case of Llama-3.1-8B, the best performing model in both setups.

One possible explanation is that SFT is inherently less effective at generalization. SFT is trained only on examples labeled as correct, whereas DPO uses preference pairs that contain both *preferred* and dis*preferred* examples. This setup may help DPO models better capture subtle distinctions in reasoning and code logic, as they learn to avoid plausible but incorrect completions. In contrast, SFT models may simply learn to mimic correct examples, without learning the decision boundaries that distinguish them from incorrect ones. As a result, SFT might improve syntax and structure but not reasoning quality.

This highlights an important distinction between surface level correctness (syntax, runtime, and structure) and deeper semantic correctness. While SFT improves the former, DPO appears to contribute more to the latter, which is crucial in tasks requiring reasoning and inference.

CHAPTER 5

Analysis

This chapter presents a detailed analysis of the experimental results, highlighting the strengths and limitations of the approaches explored throughout the study. We begin by analyzing the Single-pair vs All-pair Preference Dataset. Following this, we evaluate the relative effectiveness of DPO and SFT, shedding light on their generalization capabilities and behavior under different evaluation settings. Then, we try to address the DPO problem by developing a Llama-specific preference dataset, designed to address the weaknesses of the best-performing model, Llama3.1-8B. We then contextualize our findings by comparing them with the state-of-the-art system ViperGPT, and discuss how differences in model scale and VLM selection influence performance.

Furthermore, we examine the observed discrepancy between validation and test set accuracies. The chapter also provides a preliminary estimation of semantic versus inference errors, offering insights into the nature of model mistakes. Finally, we investigate the notion of collective versus individual knowledge, and assess whether fine-tuning strategies are effective in approximating the collective upper bound.

5.1 Single-pair vs All-pair Preference Dataset

After several fine-tuning runs, it was observed that the all-pair preference dataset yielded slightly better results than the single-pair dataset. It is important to note that the all-pair preference dataset is approximately seven times larger than the single-pair dataset. Therefore, to enable a fair comparison between training runs, the single-pair dataset would require training for significantly more epochs. Despite this, the loss metric on the development set showed better convergence with the all-pair preference dataset. Ultimately, the all-pair preference dataset can be viewed as an augmented version of the single-pair dataset. As is widely known, data augmentation generally helps improve the generalization and performance of machine learning models, especially when

dealing with limited or imbalanced datasets, which was the case here. Consequently, the all-pair preference dataset was selected for the final training phase.

For this dataset, it was observed that two epochs were more than sufficient for the training to converge. In fact, early stopping, based on the development set loss metric, had to be implemented. In many runs, the development loss would begin to increase substantially even before the completion of the first epoch. This was a point of concern, as it suggested the model was not fully leveraging all the new, unseen examples. This observation led to the consideration of reducing the learning rate in order to allow the model to train on the majority of the instances before overfitting.

Furthermore, this preference learning task appeared to be relatively easy for the model to memorize. A significant drop in the training loss was often observed right at the beginning of the second epoch, followed by a corresponding increase in the development loss. This pattern indicated that the model might be overfitting to the training data rather than generalizing effectively.

5.2 DPO and SFT effectiveness

Although DPO and SFT are not directly comparable, since they were applied to different baseline models, their performance was observed to be remarkably similar. SFT achieved larger improvements on the validation set, whereas DPO yielded higher gains on the testdev set. This suggests that DPO training may have led to better generalization performance. Notably, the highest accuracy on the testdev set was attained by the DPO fine-tuned Llama3.1-8B model.

Initially, we expected DPO to perform worse because we did not know the learning signal will be good enough. Specifically, we could not ensure that the *rejected* samples were always worse than the *preferred* ones. In some cases, the unpreferred completions may have followed a correct reasoning process, but the evaluation framework failed to recognize it, especially we could not distinguish between semantic and inference errors. This ambiguity could potentially misguide the preference signal.

It is possible that DPO increases the probability of some sequences (in this case, we refer to entire programs) that were already very unlikely for the base model. Even if their probability becomes higher, it may still be too low for the model to actually generate them. In that case, the model's ability to generate would not really change. This may happen because we are not always using examples that the base model would naturally create. Instead, we use examples from other models. As explained in Section 2.3.2, the completions y_w, y_l might be sampled from π_{ref} for each prompt x. With this idea in mind, we created the specific dataset for Llama3.1-8B, which we analyze in Section 5.3.

In contrast, SFT directly changes the probability of the most likely tokens because we are auto-regressively modifying the probability distribution for the next token using teacher forcing. This way, we replace the model's original tokens with gold-standard tokens, thus modifying the generation capabilities of the model.

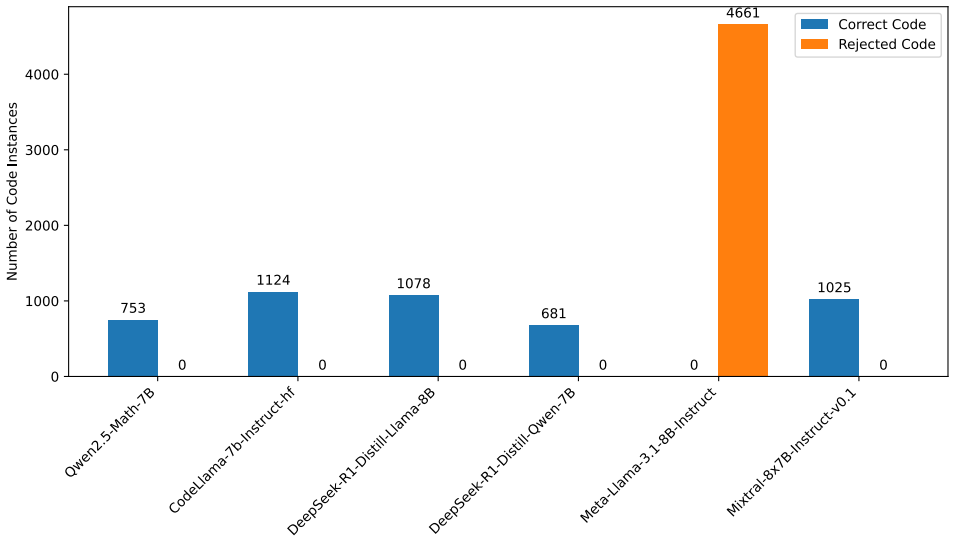

Figure 5.1: Distribution of *preferred* and *rejected* codes per model in the Llama specific dataset.

5.3 Llama Specific Preference Dataset

As noted in Section 3.1.1, LLAMA3.1-8B-IT stands out with the highest number of *preferred* codes and the lowest number of *rejected* ones. This insight motivated the construction of a third dataset specifically tailored to enhance the performance of the best performing model, LLAMA3.1-8B. For this purpose, a new preference dataset has been created containing only the codes that LLAMA3.1-8B-IT generated incorrectly. The idea is to provide the model with a targeted learning signal to discourage it from repeating its past mistakes while integrating knowledge from other models.

To construct this dataset, we first filtered out the 1000 visual questions that comprise the development set. From the remaining questions, we selected those for which LLAMA3.1-8B-IT produced an incorrect code. Notably, this subset is relatively small, considering that LLAMA3.1-8B-IT generated correct code for 6228 out of the 8431 total instances.

For each of the selected instances (those not in the development set and where LLAMA3.1-8B-IT failed), we created all possible preference pairs. In each pair, the incorrect code from LLAMA3.1-8B-IT was designated as the *rejected* code, while a correct code generated by a different model for the same visual question was chosen as the *preferred* code. This can be seen in Figure 5.1, which illustrates the distribution of *preferred* and *rejected* codes across different models. The *preferred* codes are those generated by the other models, while the *rejected* codes are those generated by LLAMA3.1-8B-IT.

This procedure resulted in a dataset of 4661 preference pairs, which can be used to further train or fine-tune LLAMA3.1-8B through DPO.

However, when we trained on this dataset, we noticed that the model overfitted very quickly. The development loss started to increase right away. Since all the *rejected* codes came from Llama3.1-8B-It, DPO was penalizing sequences that the model itself had produced. This could have made the learning signal too strong, possibly changing the model's probability distribution too much and causing catastrophic forgetting.

We tried to reduce the learning rate in several experiments, but we could not make the training stable. Because the development loss kept increasing, it made no sense to select a checkpoint to evaluate code generation. For this reason, we decided not to use this dataset to train the model.

5.4 Comparison with ViperGPT

Our best result on the testdev set was 47.8% accuracy, achieved with the Llama3.1-8B$_{\text{DPO}}$ model. When compared to ViperGPT, we can see that we almost matched their 48.1% accuracy on the same set (see Table 4.3 and Table 4.4). This is a strong result, especially considering that they used a much larger model, `code-davinci-002`, which has 175 billion parameters, while our models had only 7 to 8 billion.

In addition, the VLMs used by ViperGPT were more powerful: GLIP-L (large), X-VLM fine-tuned for retrieval on MSCOCO, and BLIP-2 Flan-T5 XXL. In contrast, we used BLIP-2 Flan-T5 XL, a smaller model that fits on a single 32GB GPU. The performance gap between these vision models is not very large. BLIP-2 Flan-T5 XXL achieves 44.7% accuracy on the GQA testdev set, while BLIP-2 Flan-T5 XL reaches 44.2% on the same set. However, because ViperGPT relies heavily on this specific VLM, this difference may still have a noticeable effect.

5.5 Accuracy decrease from validation to test set

The accuracy downgrade of the results from the validation set to the test set was about a 10% for the instruction-tuned models and 8% for the base models (compare Table 4.3 and 4.4 with Table D.1 and Table D.2 in the Appendix D). This unexpected drop also reduced the apparent improvements of the fine-tuned models over the original ones. For example, Qwen2.5-7B-It improved a 7.7% on the validation set, but just 4.82% on the testdev set, even without seeing any instance from neither of the sets. An explanation for this could be that the VLMs used by ViperGPT were fine-tuned using the validation set, hence they performed better on it. This indicates the high reliance of the framework on the VLMs, even though the code generator induced a correct reasoning process the results could not be correct. This is something we cannot know because seeing whether the error comes from the program generator or the VLMs is not possible without human evaluation or other techniques like LLM as a judge.

5.6 Semantic and inference errors estimation

To better understand the nature of the incorrect answers produced by the models, we conducted a manual evaluation aimed at distinguishing between semantic and inference errors. A random sample of 40 instances was drawn from the codes generated by LLAMA3.1-8B$_{\text{DPO}}$ that were classified as semantic or inference errors by the evaluation pipeline. Each sample was examined manually to determine whether the error was due to incorrect reasoning logic in the generated code, or whether the program itself was valid but produced an incorrect answer because of a misprediction by a VLM. The results showed a clear predominance of inference related failures. Only 10% of the errors were due to incorrect program generation, while the remaining 90% were attributable to inference failures during execution.

However, in 60% of the analyzed samples, the generated codes were counted as semantically correct, but compositional reasoning was not applied. For example, for the query:

"What color is the dirt?"

Listing 5.1: Non-compositional generated code

```
def execute_command(image):
    image_patch = ImagePatch(image)
    return image_patch.simple_query("What␣color␣is␣the␣dirt?")
```

This code directly forwards the query to a single VLM via a `simple_query` call, bypassing any form of compositional or step-wise reasoning. Such direct queries may work for some questions, but are the main source of errors in our approach. It is important to note that the version of BLIP used in our setup achieves only 44.2% accuracy in the testdev set, so relying solely on it to answer complex queries is insufficient.

Nevertheless, we also found examples of generated programs with correct compositional logic that still failed to produce the right answer due to errors introduced by the VLMs. For example, given the query:

"Where is the skinny person standing?"

Listing 5.2: Compositional code example with intermediate reasoning

```
def execute_command(image):
    image_patch = ImagePatch(image)
    person_patches = image_patch.find("person")

    # Question assumes only one person patch
```

```
    if len(person_patches) == 0:
        # If no person is found, query the image directly
        return image_patch.simple_query("Where is the skinny person standing?")

    for person_patch in person_patches:
        if person_patch.simple_query("Is the person skinny?") == "yes":
            return person_patch.simple_query("Where is the skinny person standing?")

    # If no skinny person is found, pick the first person
    return person_patches[0].simple_query("Where is the skinny person standing?")
```

This code exhibits compositional reasoning by first locating all "person" instances in the image and then verifying whether each candidate satisfies the "skinny" attribute. Only after identifying a suitable candidate does it proceed to determine the spatial relation of the subject. If no such instance is found, it resorts to a fallback strategy.

5.7 Collective vs Individual knowledge

Table 5.1 shows the distribution of *testdev* instances across different evaluation patterns. These patterns are the same as those used in Table 3.2, but here we evaluate, on the one hand, the six models used in our experiments (see Section 4.1.1 and 4.1.2) without any additional fine-tuning (*Not Fine-tuned*), and on the other hand, the same models after applying DPO on the base models and SFT on the instructed models (*Fine-tuned*).

This table is not directly comparable to Table 3.2, since the latter was computed using results from the training set, whereas this one is based on the *testdev* set. It would have been interesting to also evaluate these models on the training set to observe how much their performance improved and if they surpass the upper bound defined by the collective knowledge of all models. This would allow us to create a new preference dataset and repeat the process with more instances.

If we sum the number of instances corresponding to patterns A–H in the *Not Fine-tuned* column, we obtain a total of 8082 instances out of 12578, which corresponds to an accuracy of $8082/12578 \approx 0.64$. This result reflects the collective knowledge of all models. If we could select the correct model for each instance, we could achieve up to 64% accuracy on the *testdev* set.

Summing the same patterns (A–H) in the *Fine-tuned* column yields 8060 instances, which results in an accuracy of $8060/12578 \approx 0.64$, slightly lower than before. At first glance, this might seem disappointing. However, it is important to understand that what we are measuring here is the overall knowledge across all models. Our fine-tuning process does not explicitly aim to increase this collective knowledge (although it may do so implicitly, if generalization occurs). Instead, it aims to bring each model's individual knowledge closer to the collective knowledge of the group. In other words, to reduce the gap between each model's local knowledge and the collective knowledge shared by the group. This can be intuitively understood as changing the

Pattern	Compilation Error	Runtime Error	Semantic or Inference Error	Correct	Instances (Not fine-tuned)	Instances (Fine-tuned)
A				✓	2039	2693
B			✓	✓	4679	4652
C		✓	✓	✓	679	349
D	✓	✓	✓	✓	35	2
E	✓		✓	✓	230	20
F		✓		✓	319	337
G	✓	✓		✓	11	0
H	✓			✓	90	7
I			✓		3656	4024
J		✓	✓		532	469
K	✓	✓	✓		30	4
L	✓		✓		278	21
M		✓			0	0
N	✓	✓			0	0
O	✓				0	0
				Total	12578	12578

Table 5.1: Distribution of the 12578 visual question instances across patterns in the not fine-tuned and fine-tuned model evaluations.

generated codes so that the instances now fall in a higher, better pattern.

Explicitly, what we achieve with this training is that instances move upward, but without changing color group. Moving to a different color group (from yellow to green) would imply that generalization is occurring, since it would mean we are now getting the correct reasoning for an instance that before none of the models could solve.

In Section 3.1, we posed the open question of whether a model could generalize during training well enough to surpass the upper bound defined by the collective knowledge of all models. However, as these results show, this has not been the case in our current experiment, at least when analyzing the total number of instances falling into the green patterns (A–H). It has to be clear that we have trained with instances of the training set and then evaluate the original and fine-tuned models on the *testdev* set, so we are not sure whether the upper bound of the training set has been reached or not.

To better understand how instances transition between patterns, we refer to Figure 5.2, which illustrates how training affected the distribution of instances across evaluation patterns. Diagonal cells indicate instances that remained in the same pattern, while off-diagonal cells show how many instances moved from pattern i to pattern j. What we aim for is a greater number of instances below the diagonal ($i > j$) than above ($i < j$), which would indicate that instances have shifted to higher (better) patterns. We can observe that this is indeed the case, most instances have moved

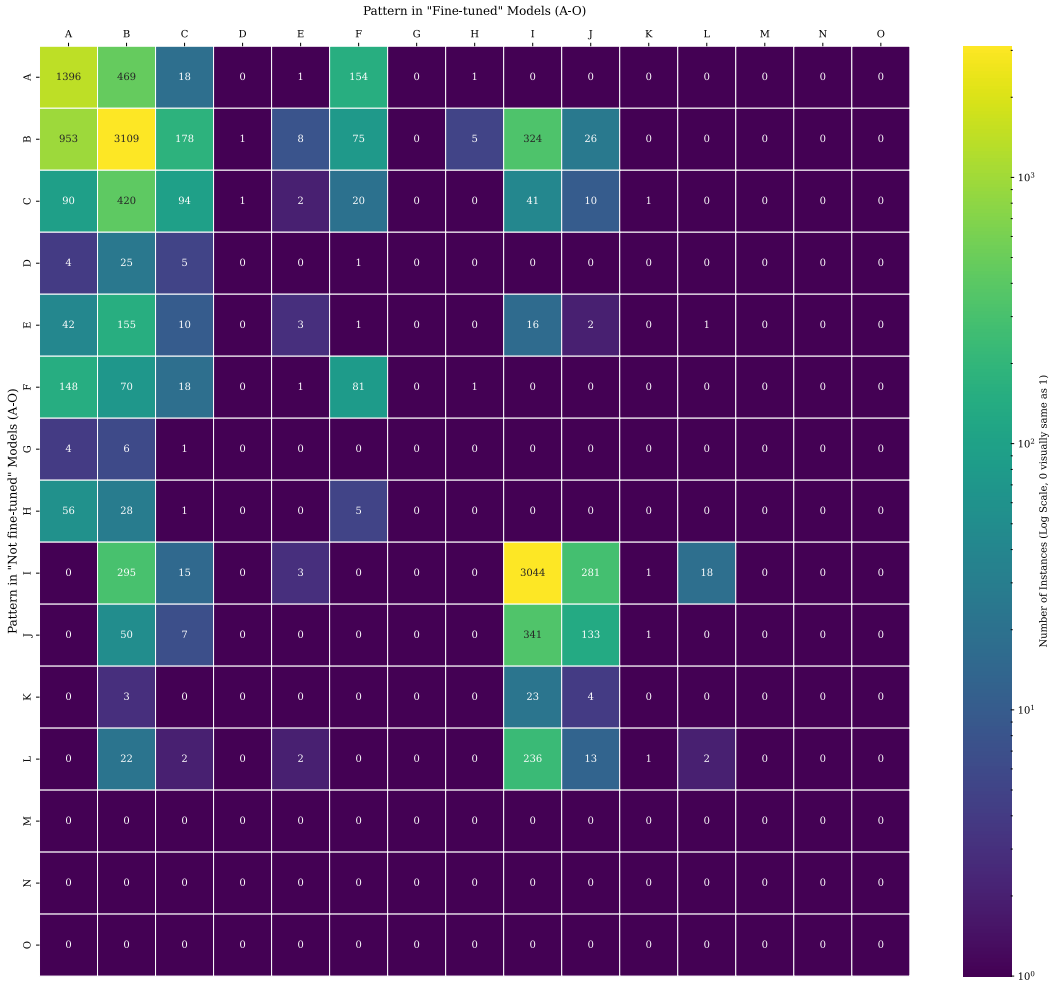

Figure 5.2: Transition matrix that shows how the training affected where the generated codes fall in the evaluation patterns. The cell i,j shows the number of instances that were in pattern i before training and moved to pattern j after training. The diagonal cells show the number of instances that remained in the same pattern.

to higher patterns (3068 instances), though some have shifted downward (2078 instances). This suggests that training had a positive effect on the majority of instances, though not on all.

We can also observe that some instances change color group. For example, 295 instances moved from pattern I to B. However, there are many other cases where instances move from a green pattern to a lower one, slightly more, in fact. Therefore, we cannot claim that fine-tuning has led to a generalization of knowledge.

It is striking that although the models together can correctly solve 64% of the instances, it

remains challenging for a single model to achieve strong performance on its own. This indicates that the proposed approach has considerable room for improvement. It is likely that further training and improved datasets could increase individual model performance, potentially up to the 64% collective upper bound. Moreover, if we observe that collective knowledge improves on the training set (a direction for future work), this process could be repeated iteratively: generating new preference datasets, retraining the models, and evaluating them on the *testdev* set. A reasonable stopping criterion would be when the collective accuracy of all models becomes the same to that of individual models.

An additional factor that might be influencing the variability in results is the possible non-deterministic behavior of the VLMs. So far, we have implicitly assumed that their outputs are deterministic; however, this is not necessarily the case. For instance, the temperature parameter in BLIP is set to 1, which introduces randomness into the generated responses and could explain part of the observed variance.

CHAPTER 6

Conclusions and future work

6.1 Conclusions

This thesis addressed a significant limitation in code-based visual reasoning frameworks like ViperGPT: the inability to further adapt the code generators. The primary objective was to develop and implement a methodology to fine-tune these models, thereby enhancing their reasoning and code generation capabilities for specific downstream tasks, in our case GQA. The work successfully demonstrates that adapting smaller open-source models not only improves their performance but also makes them competitive with much larger proprietary systems.

The foundational contribution of this work was the creation of two synthetic datasets derived from the GQA train set. The first, a preference dataset, was designed for DPO, containing pairs of *preferred* and *rejected* code generations. The second was a dataset of gold-standard programs suitable for SFT, containing code verified to produce correct answers to visual queries (further details can be found in Section 3.1).

Through extensive experimentation, this work successfully demonstrated that fine-tuning significantly improves the performance of open-source LLMs within the ViperGPT framework. Both DPO and SFT methodologies achieved a reduction in syntax, runtime, and semantic errors while increasing the rate of correct answer generation. While SFT showed stronger performance on the validation set, DPO proved to be more robust, exhibiting superior generalization to the more challenging testdev partition. This suggests that learning from both good and bad examples, as DPO does, allows the model to generalize its understanding to new problems better than simply copying correct answers.

Notably, the fine-tuned models, with only 7–8B parameters, achieved performance competitive with the original 175B model used in ViperGPT, reaching an accuracy of 47.80% on the GQA testdev set compared to ViperGPT's 48.1% (results can be found in Section 4.2). This result underscores the

efficacy of the proposed adaptation methodology in creating highly efficient and capable models. A close look at the mistakes the models still made showed that the main problem was often not the code itself, but the vision models it uses to process the image. Manual evaluation showed that about 90% of these mistakes came from the VLMs, not from bad logic in the generated code. However, even though the logic was sound, it was sometimes too vague, many times, the code simply forwarded the question directly to the vision model, relying on it to infer the answer without enough guidance or constraints.

Finally, this thesis looked at the idea of a collective and individual knowledge. It showed that the training successfully moves the shared knowledge into a single model. However, after training many of the models, the shared knowledge did not improve (see Section 5.7), so no generalization was achieved at all during training.

6.2 Future Work

This work opens up several promising directions for future research, both in terms of model adaptation strategies and dataset construction. Below, we outline some of the most relevant paths worth exploring:

Iterative data refinement. Although the adapted models did not significantly improve shared knowledge on the testdev set, this does not imply the same for the training set. It is possible that the fine-tuned models (via DPO or SFT) did some generalizations and improved some instances within the training set that previously were not correct for none of the models. If this is the case, an iterative approach could be applied: re-generate code samples for the training questions using the adapted models. These new generations may exhibit better performance and a higher collective upper bound, thus yielding larger, higher-quality preference datasets for subsequent rounds of training.

Combining SFT and DPO. The models in this work were trained using either SFT or DPO, but not both. A natural extension would be to explore a hybrid training pipeline that first uses SFT to expose the model to high-quality reference programs and then refines it with DPO to further align preferences based on pairwise comparisons. This combination may leverage the strengths of both methods: SFT for grounding the model in syntactically and semantically correct patterns, and DPO for refining preference-based generation behavior. This would resemble the traditional multi-stage training paradigm commonly used in building LLMs, where supervised fine-tuning is followed by preference-based alignment.

Scaling the Training Dataset. For computational reasons, only 12600 instances from the GQA training set were used to train the models, despite the full training set containing over 943000 questions. Expanding the training dataset would allow for greater diversity of examples and more opportunities for the model to learn from shared knowledge. Even if the percentual difference

between local knowledge and shared knowledge remains constant across a larger dataset, the absolute difference of instances would increase. Therefore, the few instances that a model might be doing wrong while one of the other models is doing right, would be substantially increased.

Scaling Model Diversity and Size. Future work could benefit from scaling both the diversity and capacity of the models involved. Increasing the number of base models used during preference dataset construction would introduce a wider range of reasoning strategies and coding styles. This would help raise the theoretical collective upper bound of instances where at least one model obtains the correct answer and would also enrich the preference signal available for fine-tuning. At the same time, leveraging larger model variants may yield performance gains, in line with the scaling laws of deep learning. Combining greater diversity and size would allow us to evaluate the framework's effectiveness under more capable and varied modeling conditions.

Full Fine-Tuning vs. Parameter-Efficient Methods. In this study, LoRA was used for efficiency in fine-tuning. However, it is still an open question whether full model fine-tuning could yield better performance, particularly for tasks involving subtle shifts in reasoning or token distribution. A comparative study between LoRA and full fine-tuning could shed light on the trade-offs between performance and computational cost.

Bibliography

[1] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019. See pages vii, 18, and 19.

[2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning, 2022. See page 5.

[3] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19730–19742. PMLR, 23–29 Jul 2023. See pages 5, 10, and 11.

[4] Shaohan Huang, Li Dong, Wenhui Wang, Yaru Hao, Saksham Singhal, Shuming Ma, Tengchao Lv, Lei Cui, Owais Khan Mohammed, Barun Patra, Qiang Liu, Kriti Aggarwal, Zewen Chi, Johan Bjorck, Vishrav Chaudhary, Subhojit Som, Xia Song, and Furu Wei. Language is not all you need: Aligning perception with language models, 2023. See page 5.

[5] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. See page 5.

[6] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning, 2023. See pages 5, 6.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish,

Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. See page 6.

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. The llama 3 herd of models, 2024. See page 8.

[9] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023. See page 8.

[10] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts, 2024. See page 9.

[11] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. See page 9.

[12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. See page 10.

[13] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 12888–12900. PMLR, 17–23 Jul 2022. See page 10.

[14] Yan Zeng, Xinsong Zhang, and Hang Li. Multi-grained vision language pre-training: Aligning texts with visual concepts, 2022. See pages 11, 12.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. See page 11.

[16] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. See page 11.

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. See page 11.

[18] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, Kai-Wei Chang, and Jianfeng Gao. Grounded language-image pre-training, 2022. See pages 12, 13.

[19] Ranjay Krishna, Yuke Zhu, Oliver Groth, Justin Johnson, Kenji Hata, Joshua Kravitz, Stephanie Chen, Yannis Kalantidis, Li-Jia Li, David A. Shamma, Michael S. Bernstein, and Fei-Fei Li. Visual genome: Connecting language and vision using crowdsourced dense image annotations, 2016. See page 13.

[20] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. See pages 15, 16.

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. See page 15.

[22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. See page 16.

[23] Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. Ok-vqa: A visual question answering benchmark requiring external knowledge, 2019. See page 17.

[24] Licheng Yu, Patrick Poirson, Shan Yang, Alexander C. Berg, and Tamara L. Berg. Modeling context in referring expressions, 2016. See page 17.

Appendix

A Code availability

For the scripts and code used in this thesis, please refer to the following GitHub repository: https://github.com/JosuBarru/VisualReason-CodeGen. Here you will find the results, datasets, and scripts used for the experiments.

B GQA Prompt

Listing 1: GQA prompt used for the experiments.

```
1
2   from PIL import Image
3 from vision_functions import find_in_image, simple_qa, verify_property,
      best_text_match
4
5 def bool_to_yesno(bool_answer: bool)->str:
6     return "yes" if bool_answer else "no"
7
8 class ImagePatch:
9     """A Python class containing a crop of an image centered around a particular
      object, as well as relevant information.
10    Attributes
11    ----------
12    cropped_image : array_like
13        An array-like of the cropped image taken from the original image.
14    left : int
15        An int describing the position of the left border of the crop's bounding box
      in the original image.
16    lower : int
17        An int describing the position of the bottom border of the crop's bounding box
       in the original image.
18    right : int
```

```
        An int describing the position of the right border of the crop's bounding box in the original image.
    upper : int
        An int describing the position of the top border of the crop's bounding box in the original image.

    Methods
    -------
    find(object_name: str)->List[ImagePatch]
        Returns a list of new ImagePatch objects containing crops of the image centered around any objects found in the image matching the object_name.
    simple_query(question: str=None)->str
        Returns the answer to a basic question asked about the image. If no question is provided, returns the answer to "What is this?".
    exists(object_name: str)->bool
        Returns True if the object specified by object_name is found in the image, and False otherwise.
    verify_property(property: str)->bool
        Returns True if the property is met, and False otherwise.
    best_text_match(string1: str, string2: str)->str
        Returns the string that best matches the image.
    crop(left: int, lower: int, right: int, upper: int)->ImagePatch
        Returns a new ImagePatch object containing a crop of the image at the given coordinates.
        """

    def __init__(self, image, left: int=None, lower: int=None, right: int=None, upper: int=None):
        """Initializes an ImagePatch object by cropping the image at the given coordinates and stores the coordinates as attributes.
        If no coordinates are provided, the image is left unmodified, and the coordinates are set to the dimensions of the image.
        Parameters
        -------
        image : array_like
            An array-like of the original image.
        left : int
            An int describing the position of the left border of the crop's bounding box in the original image.
        lower : int
            An int describing the position of the bottom border of the crop's bounding box in the original image.
        right : int
            An int describing the position of the right border of the crop's bounding box in the original image.
        upper : int
```

```
        An int describing the position of the top border of the crop's bounding box in the original image.

    """
    if left is None and right is None and upper is None and lower is None:
        self.cropped_image = image
        self.left = 0
        self.lower = 0
        self.right = image.shape[2]  # width
        self.upper = image.shape[1]  # height
    else:
        self.cropped_image = image[:, lower:upper, left:right]
        self.left = left
        self.upper = upper
        self.right = right
        self.lower = lower

    self.width = self.cropped_image.shape[2]
    self.height = self.cropped_image.shape[1]

    self.horizontal_center = (self.left + self.right) / 2
    self.vertical_center = (self.lower + self.upper) / 2

def find(self, object_name: str)->List["ImagePatch"]:
    """Returns a new ImagePatch object containing the crop of the image centered around the object specified by object_name.
    Parameters
    -------
    object_name : str
        A string describing the name of the object to be found in the image.
    """
    return find_in_image(self.cropped_image, object_name)

def simple_query(self, question: str=None)->str:
    """Returns the answer to a basic question asked about the image. If no question is provided, returns the answer to "What is this?".
    Parameters
    -------
    question : str
        A string describing the question to be asked.

    Examples
    -------

    >>> # Which kind of animal is not eating?
    >>> def execute_command(image)->str:
```

```
        >>>     image_patch = ImagePatch(image)
        >>>     animal_patches = image_patch.find("animal")
        >>>     for animal_patch in animal_patches:
        >>>         if not animal_patch.verify_property("animal", "eating"):
        >>>             return animal_patch.simple_query("What kind of animal is
    eating?") # crop would include eating so keep it in the query
        >>>     # If no animal is not eating, query the image directly
        >>>     return image_patch.simple_query("Which kind of animal is not eating?")

        >>> # What is in front of the horse?
        >>> # contains a relation (around, next to, on, near, on top of, in front of,
    behind, etc), so ask directly
        >>> return image_patch.simple_query("What is in front of the horse?")
        >>>
        """
        return simple_qa(self.cropped_image, question)

    def exists(self, object_name: str)->bool:
        """Returns True if the object specified by object_name is found in the image,
    and False otherwise.
        Parameters
        -------
        object_name : str
            A string describing the name of the object to be found in the image.

        Examples
        -------
        >>> # Are there both cakes and gummy bears in the photo?
        >>> def execute_command(image)->str:
        >>>     image_patch = ImagePatch(image)
        >>>     is_cake = image_patch.exists("cake")
        >>>     is_gummy_bear = image_patch.exists("gummy bear")
        >>>     return bool_to_yesno(is_cake and is_gummy_bear)
        """
        return len(self.find(object_name)) > 0

    def verify_property(self, object_name: str, property: str)->bool:
        """Returns True if the object possesses the property, and False otherwise.
        Differs from 'exists' in that it presupposes the existence of the object
    specified by object_name, instead checking whether the object possesses the
    property.
        Parameters
        -------
        object_name : str
            A string describing the name of the object to be found in the image.
        property : str
```

```
        A string describing the property to be checked.

    Examples
    -------
    >>> # Do the letters have blue color?
    >>> def execute_command(image)->str:
    >>>     image_patch = ImagePatch(image)
    >>>     letters_patches = image_patch.find("letters")
    >>>     # Question assumes only one letter patch
    >>>     if len(letters_patches) == 0:
    >>>         # If no letters are found, query the image directly
    >>>         return image_patch.simple_query("Do the letters have blue color?")
    >>>     return bool_to_yesno(letters_patches[0].verify_property("letters", "
blue"))
    """
    return verify_property(self.cropped_image, object_name, property)

def best_text_match(self, option_list: List[str]) -> str:
    """Returns the string that best matches the image.
    Parameters
    -------
    option_list : str
        A list with the names of the different options
    prefix : str
        A string with the prefixes to append to the options

    Examples
    -------
    >>> # Is the cap gold or white?
    >>> def execute_command(image)->str:
    >>>     image_patch = ImagePatch(image)
    >>>     cap_patches = image_patch.find("cap")
    >>>     # Question assumes one cap patch
    >>>     if len(cap_patches) == 0:
    >>>         # If no cap is found, query the image directly
    >>>         return image_patch.simple_query("Is the cap gold or white?")
    >>>     return cap_patches[0].best_text_match(["gold", "white"])
    """
    return best_text_match(self.cropped_image, option_list)

def crop(self, left: int, lower: int, right: int, upper: int)->"ImagePatch":
    """Returns a new ImagePatch cropped from the current ImagePatch.
    Parameters
    -------
    left : int
        The leftmost pixel of the cropped image.
```

```
        lower : int
            The lowest pixel of the cropped image.
        right : int
            The rightmost pixel of the cropped image.
        upper : int
            The uppermost pixel of the cropped image.
        -------
        """
        return ImagePatch(self.cropped_image, left, lower, right, upper)

# Examples of using ImagePatch
# Is there a backpack to the right of the man?
def execute_command(image)->str:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    # Question assumes one man patch
    if len(man_patches) == 0:
        # If no man is found, query the image directly
        return image_patch.simple_query("Is there a backpack to the right of the man?"
    )
    man_patch = man_patches[0]
    backpack_patches = image_patch.find("backpack")
    # Question assumes one backpack patch
    if len(backpack_patches) == 0:
        return "no"
    for backpack_patch in backpack_patches:
        if backpack_patch.horizontal_center > man_patch.horizontal_center:
            return "yes"
    return "no"

# In which part is the bread, the bottom or the top?
def execute_command(image)->str:
    image_patch = ImagePatch(image)
    bread_patches = image_patch.find("bread")
    # Question assumes only one bread patch
    if len(bread_patches) == 0:
        # If no bread is found, query the image directly
        return image_patch.simple_query("In which part is the bread, the bottom or the
    top?")
    if bread_patches[0].vertical_center < image_patch.vertical_center:
        return "bottom"
    else:
        return "top"

# What type of weather do you see in the photograph?
def execute_command(image)->str:
```

```
    image_patch = ImagePatch(image)
    return image_patch.simple_query("What type of weather do you see in the photograph?")

# Who is the man staring at?
def execute_command(image)->str:
    # asks for the predicate of a relational verb (staring at), so ask directly
    image_patch = ImagePatch(image)
    return image_patch.simple_query("Who is the man staring at?")

# What toy is wearing a shirt?
def execute_command(image)->str:
    # not a relational verb so go step by step
    image_patch = ImagePatch(image)
    toy_patches = image_patch.find("toy")
    # Question assumes only one toy patch
    if len(toy_patches) == 0:
        # If no toy is found, query the image directly
        return image_patch.simple_query("What toy is wearing a shirt?")
    for toy_patch in toy_patches:
        is_wearing_shirt = (toy_patch.simple_query("Is the toy wearing a shirt?") == "yes")
        if is_wearing_shirt:
            return toy_patch.simple_query("What toy is wearing a shirt?") # crop would include the shirt so keep it in the query
    # If no toy is wearing a shirt, pick the first toy
    return toy_patches[0].simple_query("What toy is wearing a shirt?")

# What is behind the pole?
def execute_command(image)->str:
    image_patch = ImagePatch(image)
    # contains a relation (around, next to, on, near, on top of, in front of, behind, etc), so ask directly
    return image_patch.simple_query("What is behind the pole?")

# Are there bagels or lemons?
def execute_command(image)->str:
    image_patch = ImagePatch(image)
    is_bagel = image_patch.exists("bagel")
    is_lemon = image_patch.exists("lemon")
    return bool_to_yesno(is_bagel or is_lemon)

# Is that blanket to the right of a pillow?
def execute_command(image)->str:
    image_patch = ImagePatch(image)
    blanket_patches = image_patch.find("blanket")
```

```
    # Question assumes only one blanket patch
    if len(blanket_patches) == 0:
        # If no blanket is found, query the image directly
        return image_patch.simple_query("Is that blanket to the right of a pillow?")
    for blanket_patch in blanket_patches:
        pillow_patches = image_patch.find("pillow")
        for pillow_patch in pillow_patches:
            if pillow_patch.horizontal_center > blanket_patch.horizontal_center:
                return "yes"
    return "no"

# INSERT_QUERY_HERE
def execute_command(image)->str:
```

C Datasets analysis

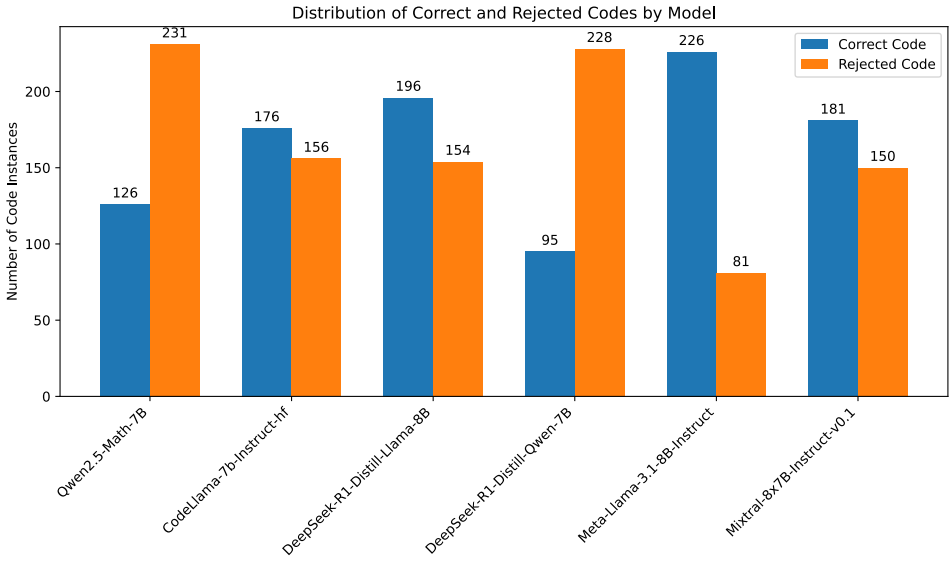

Figure C.1: Distribution of preference pairs in the development partition of the single-pair DPO dataset. This will be the dataset used as development for every approach.

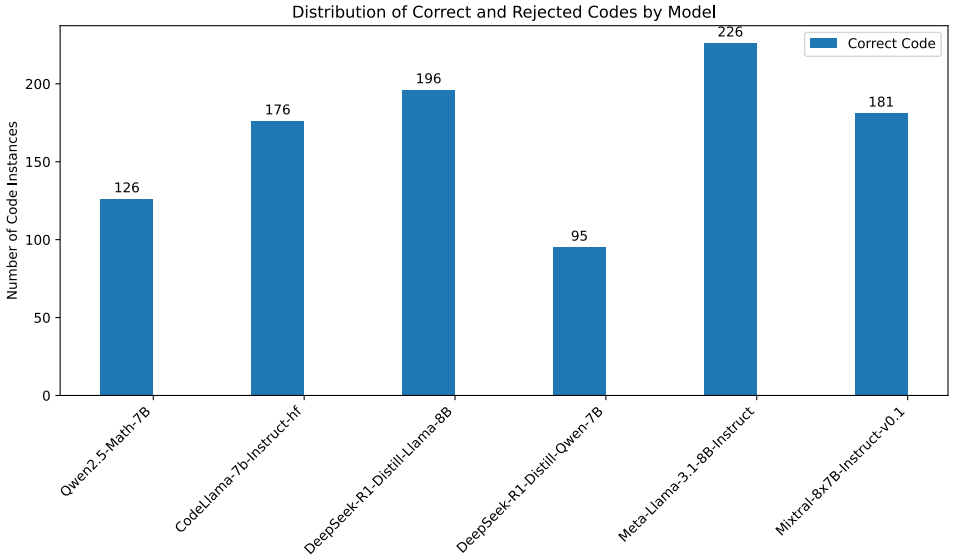

Figure C.2: Distribution of generated codes per model in the SFT dataset for the development partition.

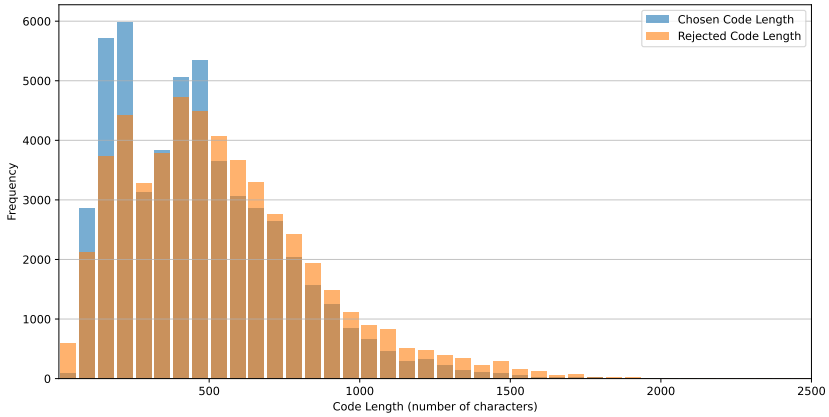

Figure C.3: Generated code length distribution comparison between chosen and *rejected* samples in every-pair approach

D Development evaluation

LLM	Syntax Error ↓	Runtime Error ↓	Semantic / Inference Error ↓	Correct ↑
Not Fine-Tuned (Baseline)				
Qwen2.5-Math-7B	0.4	10.1	45.3	44.2
CodeLlama-7B-hf	1.2	0.4	48.7	49.7
Llama-3.1-8B	1.2	0.6	45.5	52.7
Fine-Tuned Models				
Qwen2.5-Math-7B	0.2	8.4	43.4	48 (+3.8)
CodeLlama-7B-hf	0.4	0.1	44.4	55.1 (+5.4)
Llama-3.1-8B	0.1	0.1	44.1	55.7 (+3)

Table D.1: Comparison of zero-shot and DPO fine-tuned LLMs on the GQA validation partition. To evaluate our models GLIP-Large and BLIP2-Flan-T5-XL were used.

LLM	Syntax Error ↓	Runtime Error ↓	Semantic / Inference Error ↓	Correct ↑
Not Fine-Tuned (Baseline)				
Qwen2.5-7B-It	2.7	3.5	45.9	47.9
CodeLlama-7B-It-hf	0	1.3	45.6	53.1
Llama-3.1-8B-It	0.1	0.9	46.1	52.9
Fine-Tuned Models				
Qwen2.5-7B-It	0	0.1	44.3	55.6 (+7.7)
CodeLlama-7B-It-hf	0	0	43.3	56.7 (+3.6)
Llama-3.1-8B-It	0	0	43.6	56.4 (+3.5)

Table D.2: Comparison of zero-shot and Supervised fine-tuned LLMs on the GQA validation partition. To evaluate our models GLIP-Large and BLIP2-Flan-T5-XL were used.

E LoRA Hyperparameters

The following hyperparameters were used for Low-Rank Adaptation (LoRA) during model fine-tuning:

Hyperparameter	Value
LoRA rank (r)	64
Target modules	`[q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]`
LoRA scaling factor (α)	64
Dropout	0

Table E.3: LoRA hyperparameters used during fine-tuning.